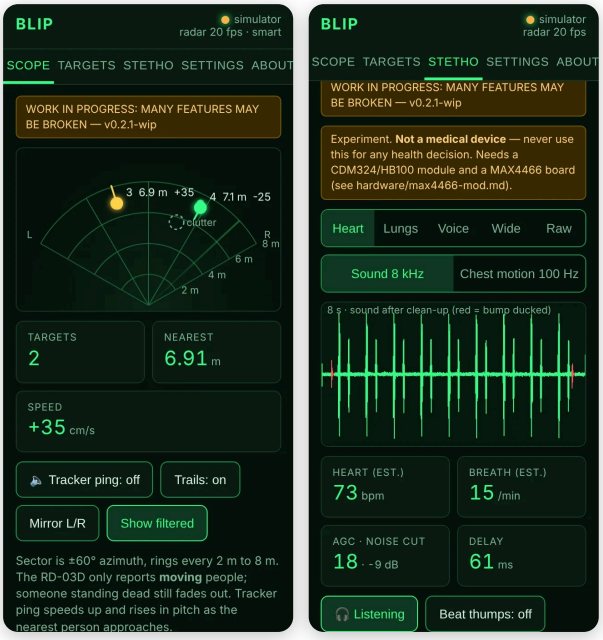
BLIP

One small ESP32, one 24 GHz radar, and an old phone as the screen.

WORK IN PROGRESS: MANY FEATURES MAY BE BROKEN

Manual and research notes · version 0.2.2-wip · 2026-11-13
https://github.com/The-Dorkknight/blip-radar



Required Notice: Copyright Matthew Armstrong (https://github.com/The-Dorkknight). Licensed under the PolyForm Noncommercial License 1.0.0 — source-available, free for non-commercial use. Provided as-is with no warranty and no liability. **Not a medical device.** Screenshots are from the simulator, not a real radar. This PDF is generated from the repository's own documents by tools/make_pdf.py.

Contents

1. Overview, setup and use — README.md
2. How it works — HOW_IT_WORKS.md
3. Design notes and research — DESIGN_NOTES.md
4. Wiring — hardware/wiring.md
5. Stetho front end: the MAX4466 mod — hardware/max4466-mod.md
6. Sound out of the node — hardware/audio-out.md
7. Changelog — CHANGELOG.md
8. Third-party notices — THIRD_PARTY.md
9. Licence — LICENSE.md

CHAPTER 1

Overview, setup and use

README.md

WORK IN PROGRESS: MANY FEATURES MAY BE BROKEN — proof of concept, v0.2.2-wip

Turn a tiny radar chip and an old phone into a people-spotting radar 📡

Ever wanted to know who is walking about, and how far away they are, without a camera? Blip does it with a tiny radar chip and a spare phone. The chip sends out invisible radio waves and listens for the echoes. When someone moves, the echo changes, and Blip works out where they are.

The phone is the screen. Blip makes its own Wi-Fi network; you join it, open a web page, and a glowing green radar scope appears, like something from a submarine film. Up to three people show up as dots with their distance, direction and speed. No app to install, no internet needed, no cloud, no camera.

How it works, in three steps 1. Plug a little ESP32 board into a radar chip (the Ai-Thinker **RD-03D**) and power it from USB. 2. Join the Wi-Fi network called *Blip-Radar* on any old phone, tablet or laptop. 3. Open the page and watch the dots move. Walk about and see yourself on the scope!

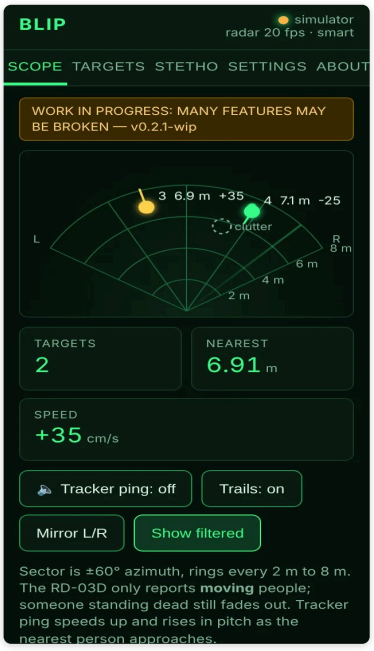
Good things to know - It only sees things that **move**. Someone sitting very still fades from the scope. - It also bounces off big metal things such as fridges, so a small room can look busy. - A tiny optional screen (an OLED) can sit on the board and show the same picture. Blip finds it by itself when it starts. - You can switch between tracking the one nearest person or up to three.

It is a slimmed-down take on Stevee87's Arduino-ESP32-Radarproject, which pairs the same radar with a XIAO ESP32-S3 *and* an Arduino GIGA R1 + display shield. The drawing is the easy part; a phone already does it.

Required Notice: Copyright Matthew Armstrong (https://github.com/The-Dorkknight). Licensed under the PolyForm Noncommercial License 1.0.0 — source-available, free for non-commercial use. Provided as-is with **no warranty and no liability**. **Not a medical device.** Radar can see motion through thin walls: respect other people's privacy and the 24 GHz rules where you live.

Project started 2026-11-05 10:24.

📘 **The whole manual and research notes as one PDF:** docs/Blip-manual.pdf (also attached to every release).



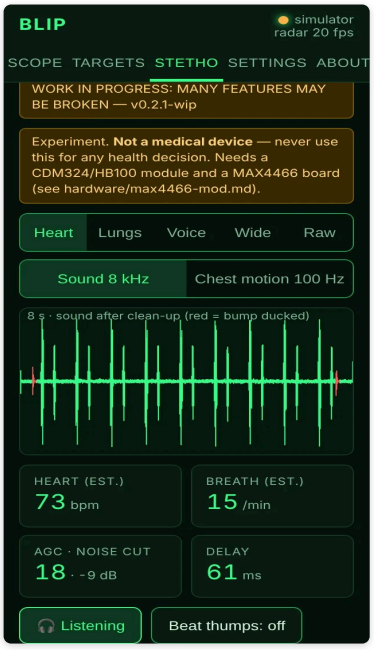
Phone screenshot from the simulator (tools/sim_radar.py).

Bonus experiment: the Stetho add-on (parked)

The idea: use a second, very cheap radar-style module (a £3 CDM324 or HB100 Doppler sensor) as a **stethoscope that listens to a heartbeat from a distance**. A modded MAX4466 microphone board boosts the signal, and the ESP32 cleans up the sound (ducks knocks, lifts heart, lung and throat sounds, cuts hiss) and plays it live to the phone, to 3.5 mm earphones, or to a small speaker.

Status: abandoned, but kept as an optional install. The radar and phone scope are the project; the stethoscope is parked. The firmware, page and tests are still in the repo and still build, and you can switch it on in Settings, but nobody is working on it. Why, in the author's words: *"Needing to build an ultra-low-noise, ultra-high-gain, fast-slew-rate instrumentation amplifier is just too much work for a 'fun' 30-minute project, but I*

> *encourage anyone gifted enough with analogue, or mad enough, to carry on my research 😉*" Findings and test recordings are in docs/STETHO_SIGNAL_NOTES.md.



Simulator screenshot. The real thing never got past the noise floor.

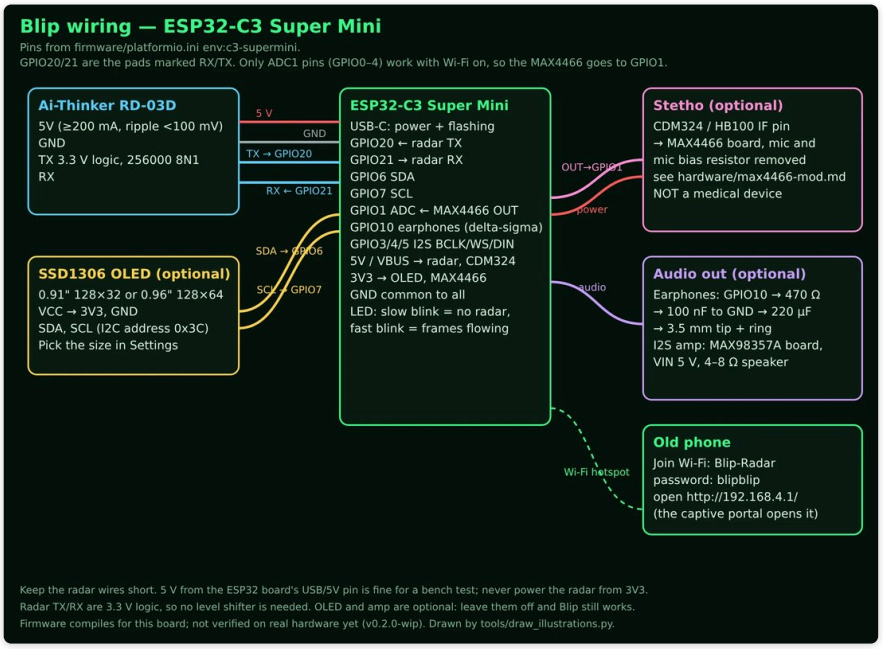
What exists vs what doesn't

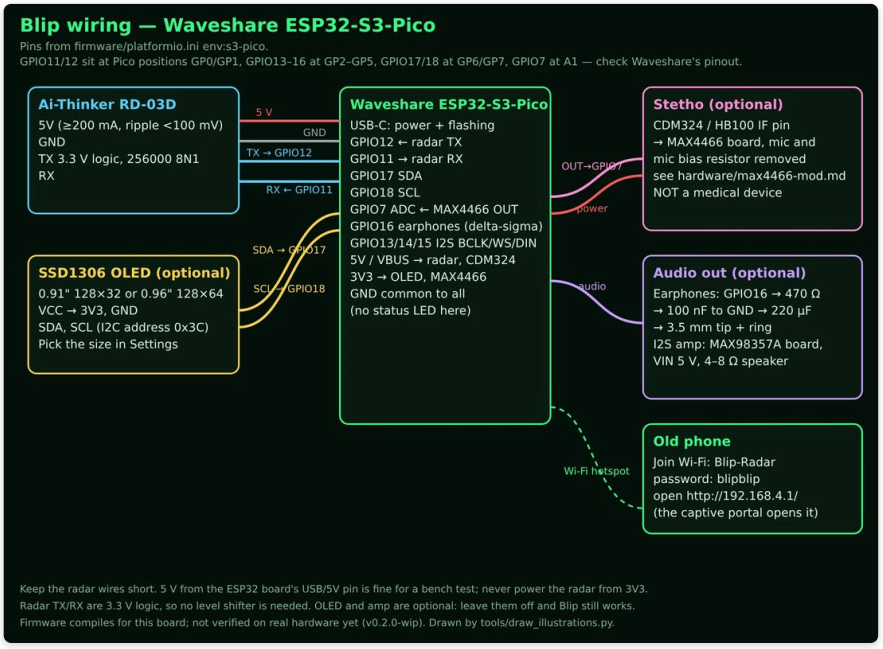
Part	State
RD-03D frame parser (sign bits, resync, ACK tolerance)	Done; unit-tested natively in C++ and Python (`tests/`)
Single/multi-target mode command	**Fixed in v0.2.1**: now wrapped in enable/end-config like the original project (earlier versions probably did nothing)
"Smart" target filter (own track ids, smoothing, hides flickers, wall reflections, fans)	**Experimental, opt-in.** Native tests on synthetic scenes pass; not tried in a real room
Auto-sleep + optional power switches for the RD-03D and the CDM324	Written, compiles; not tried on hardware
Phone web UI: scope, targets table, tracker ping, settings, about	Done; tested in Chromium against the simulator
Flasher page the launcher opens (Quick flash of bundled firmware, or build from source; serial output; error tips)	Done; tested end to end with stand-ins for esptool and PlatformIO (`tests/test_flasher.py`); the bundled images and the esptool command were checked with the real esptool (`image_info` : valid); **not yet used on a real board**
Wi-Fi hotspot + captive portal + WebSocket/HTTP API	Running on the author's bench (Nov 2026): the page loads and shows live targets
ESP32-C3 Super Mini and Waveshare ESP32-S3-Pico builds	**Compile cleanly** (pioarduino, Arduino-ESP32 3.3.12); not yet flashed
OLED mirror (0.91" 128×32 / 0.96" 128×64 SSD1306)	Works on the author's bench; found automatically at boot. The previews below are rendered by the firmware's own drawing code on a PC
Stetho audio clean-up on the ESP32 (bump gate, heart/lungs/voice presets, noise suppression, AGC)	Done in fixed point; native test on a synthetic scene passes (knocks −38 dB vs heart sounds, hiss −11 dB)
Live low-latency audio to the phone, chest-motion trace, heart/breath estimates	Done; tested in Chromium against the simulator (54–83 ms delay there)
Earphone (delta-sigma) and I2S amp outputs on the node	Written, compiles; **not heard yet**
CDM324 + modded MAX4466 front end	Built and tried on a bench (Nov 2026): static fixed by supply caps, but the heartbeat sits at the noise floor even at 2 cm. **Abandoned**: it needs a custom ultra-low-noise, high-gain instrumentation amplifier (and ideally an I/Q module)
Making the RD-03D itself into a Doppler stethoscope	**Not possible through its UART** — see DESIGN_NOTES.md
RD-03D tracking in a real room	Tracks up to 3 targets, and also big metal objects as you walk towards them; noisy in a small house, outdoor test still to do
Enclosure	None yet

Bill of materials

Qty	Part	Notes
1	ESP32-C3 Super Mini **or** Waveshare ESP32-S3-Pico	Either; both envs in `firmware/platformio.ini`
1	Ai-Thinker RD-03D — or an HLK-LD2450 (see *Radar options*)	5 V, ≥200 mA, UART 256000 baud, 3.3 V logic
1	Old phone / tablet / laptop with a browser	The display
0–1	SSD1306 I2C OLED, 0.91" 128×32 or 0.96" 128×64	Optional second display
0–1	CDM324 (24 GHz) or HB100 (10.5 GHz) CW Doppler module	Optional "Stetho"; HB100 if you want to run it alongside the RD-03D (see below)
0–2	MAX4466 electret-mic amp board (Adafruit 1063 / GY-MAX4466)	The Stetho amplifier, modded: hardware/max4466-mod.md
0–1	470 Ω, 100 nF, 220 µF, 3.5 mm socket	Earphone output, hardware/audio-out.md
0–1	MAX98357A I2S amp board + 4–8 Ω speaker	Speaker output
—	USB-C cable / 5 V supply, wire	The C3's 5V pin can feed the radar for bench tests

Wiring



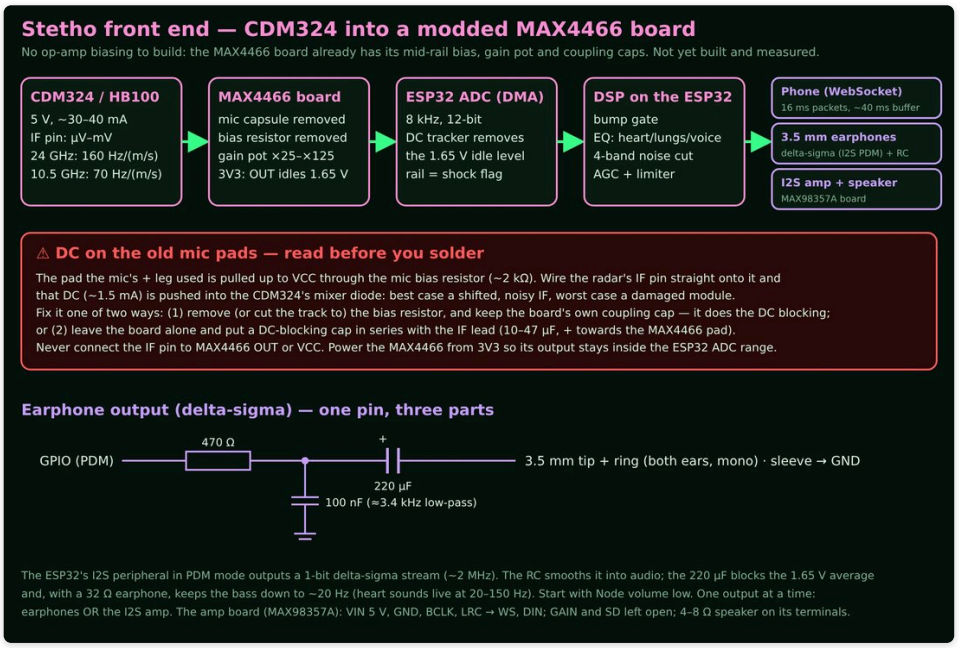


Signal	C3 Super Mini	S3-Pico	Note
Radar TX → ESP RX	GPIO20	GPIO12	UART1, 256000 8N1

Signal	C3 Super Mini	S3-Pico	Note
Radar RX ← ESP TX	GPIO21	GPIO11	only used to send the single/multi-target command
OLED SDA / SCL	GPIO6 / GPIO7	GPIO17 / GPIO18	I2C 0x3C
Stetho: MAX4466 OUT	GPIO1 (ADC1)	GPIO7 (ADC1)	MAX4466 powered from 3V3
Earphones (delta-sigma)	GPIO10	GPIO16	via 470 Ω / 100 nF / 220 µF
I2S amp BCLK / LRC / DIN	GPIO3 / 4 / 5	GPIO13 / 14 / 15	MAX98357A
Radar 5V / GND	5V / GND	VBUS / GND	never 3V3

Pin numbers live in one place, `firmware/platformio.ini`; the diagrams are drawn from it.

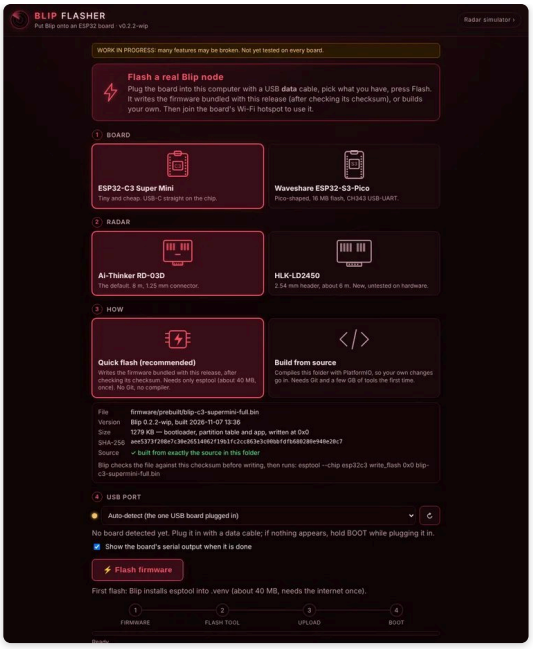
Stetho front end: CDM324 → modded MAX4466



> ⚠ **DC warning.** When you desolder the mic capsule and wire the radar's IF pin onto its pads, **also remove the mic bias resistor**, or put a 10–47 µF DC-blocking cap in series with the IF lead. Otherwise ~1.5 mA of DC from VCC is pushed into the CDM324's mixer diode. Power the MAX4466 from **3V3**, never 5 V. Full steps: hardware/max4466-mod.md.
>
> **RD-03D + CDM324 at the same time?** They share the 24 GHz band. The RD-03D's sweep passes right over the CDM324's frequency, so expect a buzz in the Stetho audio and possibly a noisier RD-03D. Use an **HB100** (10.5 GHz) for the Stetho, or fit a 5 V switch on the radar and choose *Radar off while listening* (`PIN_RADAR_PWR`). Details in DESIGN_NOTES.md §11.5.

One launcher: flash it, or try it first (double-click)

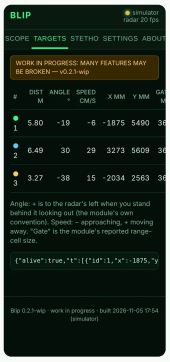
1. Unzip, then double-click `Blip-Linux.sh` (choose *Run*), `Blip-Windows.bat` or `Blip-Mac.command` (first time on a Mac: right-click → Open).
2. Your browser opens the **Blip Flasher** at `http://127.0.0.1:8080/` (the red page). Only Python 3 is needed to open it. - Want to look around before you have hardware? The small **Radar simulator ›** button (top right) opens the normal Blip page at `/sim`, with three pretend people wandering about and a synthetic Stetho signal; its ⚡ **Flasher** button brings you back. Start the launcher with `--lan` to open the simulator from your phone too (the terminal prints the address).
3. **To flash a real board:** plug the board in with a USB *data* cable, pick the board (C3 Super Mini / S3-Pico), the radar (RD-03D / LD2450) and how to flash, pick the port (or leave Auto-detect), press ⚡ **Flash firmware**. - **Quick flash (default, recommended):** writes the firmware that ships in `firmware/prebuilt/`. Nothing is hidden: the page shows the file, version, build date, size and full SHA-256, checks the file against `manifest.json` before writing, says whether it was built from exactly the source in your folder, and shows the exact esptool command. The first run installs only esptool into `.venv` (about 40 MB). No Git, no compiler. - **Build from source:** compiles your folder with PlatformIO so your own changes go in. The first build downloads the ESP32 toolchain (a few GB; needs the internet and **Git**; 5–15 minutes), later builds are quick. - The steps light up as it goes (Quick: Firmware → Flash tool → Upload → Boot; Source: Prepare → PlatformIO → Build → Upload → Boot) with a progress bar for the upload and the full log underneath (*Copy* puts it on the clipboard for a bug report). - If something fails, a tip explains the usual cause: serial-port permission on Linux, the board not in flashing mode (hold BOOT, tap RESET), no board or several boards found, no internet on the first run, Git missing, a port held by another program, your source newer than the bundled firmware. - Afterwards it shows the board's serial output (no terminal needed). *Cancel* stops any step. - Rather use another tool? Any ESP32 flasher can write `firmware/prebuilt/blip-<env>-full.bin` at address `0x0` (they are also attached to each GitHub release).
4. Wire the radar as above and power up. The LED on the C3 blinks slowly until frames arrive, then fast.
5. On the phone: join Wi-Fi `Blip-Radar`, password `blipblip`. The captive portal should open the scope; if not, browse to `http://192.168.4.1/`. It is the same green page as the simulator.

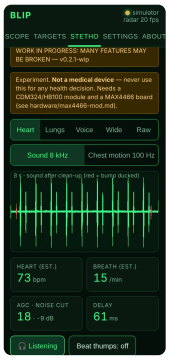
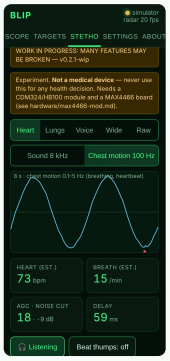
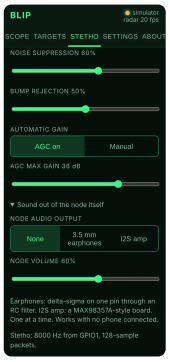
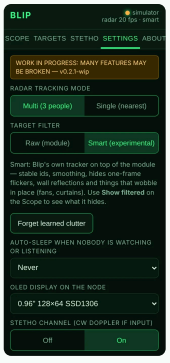
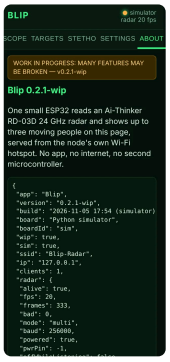


The flasher only answers this computer (`127.0.0.1` / `localhost`): starting a flash needs a per-run token only the page has, and the board, radar and port are checked against fixed lists before anything runs. `--no-flash` hides it entirely.

Prefer the terminal? `cd firmware && pio run -e c3-supermini -t upload` (envs: `c3-supermini` , `s3-pico` , `c3-supermini-ld2450` , `s3-pico-ld2450`). After editing `web/index.html` run `python3 tools/embed_web.py` (the Flash tab does this for you).

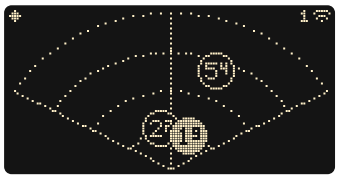
Using it

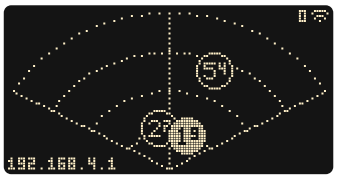
Screen	What it does
	**Scope** — ±60° sector, rings every 2 m to 8 m, one coloured blip per tracked person with distance and radial speed. Trails fade over 4 s. *Tracker ping* is the original project's buzzer, done in the phone's speaker: faster and higher as the nearest person approaches. *Mirror L/R* flips the picture if you mount the radar the other way round. With the Smart filter on, *Show filtered* draws what it is hiding as dashed rings (ghost / clutter).
	**Targets** — the numbers behind the blips plus the raw JSON, handy when checking orientation.

Screen	What it does
	**Stetho** — the optional CW Doppler channel. Presets: **Heart** (S1/S2 heart sounds, 20–150 Hz), **Lungs** (breath hiss), **Voice** (throat/larynx buzz), **Wide**, **Raw** (plain Doppler warble). *Listen on phone* plays it live; *Delay* shows how far behind real time you are. Knocks are ducked and limited (red in the trace). **Beat lock** quietens the gaps between beats once a rhythm is found; **Record 10 s → WAV** saves a recording for `tools/analyze_wav.py` ; *Experiments* has ADC averaging and the burst blanker (see docs/STETHO_SIGNAL_NOTES.md). **Not a medical device.**
	**Chest motion** view — the 100 Hz motion trace with heart-rate and breathing rates from the pattern trackers (a `?` means only roughly steady); *Beat thumps* adds a synthetic thump on each detected beat.
	**Clean-up on the node** — noise suppression, bump rejection, AGC or manual gain; **Sound out of the node** — earphones or I2S amp, with its own volume.
	**Settings** — single/multi target mode (also on the Radar page as *Track 1* / *Track 3*), **Target filter** (Raw / Smart, experimental) with *Forget learned clutter*, **Auto-sleep**, OLED (Auto-detect, or a fixed size), Stetho on/off, radar-off-while-listening, hotspot name and password (reboots).
	**About** — version, build date, board, pin map and counters.

Screen	What it does
	**Flash** (simulator on a PC only) — pick board, radar, Quick flash or Build from source, and port; see exactly what will be written; watch the board boot.

OLED mirror. Each person is a circle with their distance inside: big digit = metres, small raised digit = tenths, so "1⁹" = 1.9 m. The nearest person is solid. There's no status row, so the scope fills the panel. ◆ top-left = Smart filter on; Wi-Fi icon + number top-right = phones connected; the IP shows bottom-left only until a phone joins. These are exact renders of the firmware's drawing code, not photos:







How it works

Short version: the ESP32 parses 30-byte frames from the radar, keeps the latest three targets, and pushes them as JSON over a WebSocket 20 times a second to whoever is on the hotspot; the browser draws the scope on a canvas. The long version, including the frame format, is in HOW_IT_WORKS.md. The research — radar physics, why the RD-03D can't be a stethoscope and what can, pin choices — is in DESIGN_NOTES.md.

Radar options

Blip was written for the **Ai-Thinker RD-03D** (1.25 mm 4-pin socket, 8 m, up to 3 targets). The **HLK-LD2450** (Hi-Link, 24 GHz, about 6 m, up to 3 targets with X/Y position) is supported too:

- It sends the same 30-byte frame (`AA FF 03 00` , three 8-byte slots, `55 CC`), so the same parser reads it.
- It has no RD-03D configuration commands, so Blip skips the mode command. *Single (nearest)* is picked in firmware instead.
- Use the `c3-supermini-ld2450` or `s3-pico-ld2450` build environment (or add `-DRADAR_LD2450=1`). `/api/status` reports `"model":"ld2450"` while it is running.
- Its pins are on a 2.54 mm header, so jumper wires work. Check the labels on the module: its **TX** goes to the board's **RX** pin, and its **RX** to the board's **TX**.
- Weaker than the RD-03D on people who sit very still (the vendor notes this beyond about 2 m), and range is about 6 m.
- Still 24 GHz, so it clashes with a CDM324 in the same way. Use an HB100 for Stetho, or the radar power switch.
- **Not tested on hardware yet.** Check the angle direction on the Scope first (*Mirror L/R* in Settings if left and right are swapped).

Troubleshooting

Symptom	Try
Phone joins but no page opens	Browse to `http://192.168.4.1/` by hand; some phones silence captive portals.
"radar not sending — check 5 V and TX/RX"	Swap TX/RX (the most common mistake), confirm 5 V at the radar, baud is 256000 (not 115200).
Frames arrive but nobody shows up	The RD-03D only reports **moving** people. Walk about. Check *Mirror L/R* if left/right look swapped. With Smart on, try Raw: the filter may be hiding someone (*Show filtered* tells you why).
Phantom people / a second "you" further away	Wall or window reflections. Try Settings → Target filter → **Smart**; also mount the radar 1.2–1.5 m high and not facing glass or metal.
A real person keeps vanishing with Smart on	Two people walking in single file, in step, look exactly like a reflection; or someone fidgeting in one spot for 30 s gets muted as clutter. Press *Forget learned clutter* or use Raw.
OLED blank	The node scans for it at boot (0x3C, then 0x3D) and the Logs say what it found. If the picture is wrong or half-missing, pick the size in Settings (Auto assumes 128×64; a 0.91" panel is 128×32). Check 3V3, GND, SDA/SCL (C3: GPIO6/7, S3-Pico: GPIO17/18). SH1106 panels are not supported yet.
Stetho shows only noise	Expected without a module + MAX4466. With one: turn the MAX4466 pot up, keep the module still, subject within ~1 m, try the HB100 if the RD-03D is running.

Symptom	Try
Steady buzz in the Stetho audio	The RD-03D's 24 GHz sweep hitting the CDM324. Use an HB100 or switch the radar off while listening.
Howl / feedback from the speaker	The radar hears the speaker cone. Use earphones, point the speaker away, lower Node volume.
Phone audio lags or stutters	Check *Delay*; old phones may need the screen on. The node drops stale packets rather than lagging; "lost" counts them.
Flash script finds no port (C3)	Hold BOOT, tap RESET, release BOOT, run again. On macOS/older Windows the S3-Pico needs the CH343 driver.
Firmware doesn't compile	It builds in the author's sandbox with the pinned platform; please open an issue with the error and your PlatformIO version.

Known limitations

- The RD-03D is Doppler-based: stationary people vanish after a few seconds. Range resolution is coarse (0.75 m gates, 0.15 m accuracy claimed); angle is from a two-antenna phase comparison, so it wobbles.
- Through-wall detection exists but range drops hard; that's 24 GHz physics.
- Only 802.11 hotspot mode: the node does not join your home Wi-Fi (yet).
- No authentication on the hotspot beyond the WPA2 password.
- The RD-03D and a CDM324 share the 24 GHz band (see above).
- Neither radar has a sleep command; switching them off needs the optional power switches.
- Audio to the phone has ~60–120 ms delay; "no latency" over Wi-Fi isn't physically on offer. The node's own outputs are ~20 ms.
- Nothing here has met real hardware. Expect a round of fixes.

Repository map

```
firmware/        PlatformIO project (src/main.cpp, include/rd03d.h, tracker.h, audio_dsp.h, stetho.h, oled.h, oled_draw.h, web_ui.h)
firmware/prebuilt/  bundled whole-chip images for the four builds + manifest.json (checksums, version, source hash)
web/index.html   the phone UI (single file; embedded gzipped into the firmware)
web/flasher.html   the flasher (served by the PC tool only; never in the firmware)
web/flash_home.html  the red home page the launcher opens, with the flasher inlined
tools/           sim_radar.py, flasher.py, serial_tail.py, make_prebuilt.py, make_pdf.py, embed_web.py, draw_illustrations.py, rd03d_py.py,
tracker_py.py
tests/           native C++ tests (parser, audio DSP, tracker, OLED render), Python parser + tracker cross-check, flasher (with
fake_pio.py), launcher test
hardware/        wiring, MAX4466 mod (with the DC warning), earphone / I2S amp outputs
examples/        synthetic RD-03D frames for replay; dsp_before.wav / dsp_after.wav
docs/            GitHub Pages site + images + Blip-manual.pdf (built by tools/make_pdf.py)
```

Credits

- Stevee87's Arduino-ESP32-Radarproject (MIT) for the idea, the tracker-ping behaviour, and (from reading its v1.1 firmware) the enable/end-config wrapping of the mode command. No code was copied.
- Ai-Thinker for the RD-03D protocol document; the Electronic Clinic write-up for a captured two-target frame used in the tests.
- See THIRD_PARTY.md.

CHAPTER 2

How it works

HOW_IT_WORKS.md

Plain-English tour, then the protocol details.

What it reads, what it changes, what it never does

- **Reads:** bytes from the RD-03D's TX pin (UART1, 256000 baud). Optionally one ADC pin (the Stetho channel, 8 kHz).
- **Writes to the radar:** a three-command sequence at boot and whenever you change mode, 200 ms apart: *enable configuration* (`FD FC FB FA 04 00 FF 00 01 00 04 03 02 01`), *single-target* (`FD FC FB FA 02 00 80 00 04 03 02 01`) or *multi-target* (`… 90 00 …`), then *end configuration* (`FD FC FB FA 02 00 FE 00 04 03 02 01`). v0.2.0 and earlier sent the mode command alone, which the module probably ignored. Nothing else is written; the radar's other settings are never altered.
- **Stores:** a handful of settings in the ESP32's NVS flash: mode, target filter, auto-sleep time, OLED size, Stetho on/off and its clean-up/output settings, hotspot name and password. The Smart filter's clutter map lives in RAM only and is forgotten at power-off.
- **Never does:** connect to the internet, join your home Wi-Fi, record or log positions or audio, or send anything to anyone but the phone that joined its hotspot.

The pieces

```
RD-03D ──UART 256000──► ESP32 ──Wi-Fi hotspot 192.168.4.1──► phone browser
                        │  ├─ /            the UI (gzipped HTML in flash)
                        │  ├─ /ws          WebSocket: targets JSON 20/s, Stetho audio 62/s + chest motion 10/s
                        │  ├─ /api/status  JSON: version, board, pins, counters, settings
                        │  ├─ /api/targets JSON: same as the WebSocket text frames (polling fallback)
                        │  └─ /api/config  ?mode= &filter=raw|smart &forget=1 &sleep=min &oled= &stetho=
                        │                   &preset= &ns= &bump= &agc= &maxgain= &gain= &out= &ovol=
                        │                   &radarOff= &ssid= &pass=
                        ├─ optional SSD1306 OLED mirror (I2C; found by a scan at 0x3C/0x3D at boot)
                        ├─ optional Stetho: ADC DMA 8 kHz → DSP task → phone / earphones / I2S amp
                        └─ optional power switches: RD-03D (PIN_RADAR_PWR), CDM324 (PIN_STETHO_PWR)
```

Everything runs in the Arduino `loop()` except Wi-Fi/TCP (handled by ESPAsyncWebServer's own tasks) and the Stetho audio task (priority 5), which wakes every 64 samples from the ADC's DMA, runs the DSP, writes the I2S output and queues packets that `loop()` sends. On the single-core C3 this is fine because the loop never blocks: the UART parser is fed byte by byte, the OLED redraws at most 10 times a second, and WebSocket sends are dropped when nobody is listening or the TCP buffer is full.

The RD-03D frame

Every report is exactly 30 bytes, little-endian:

```
AA FF 03 00                              header
x_lo x_hi  y_lo y_hi  v_lo v_hi  r_lo r_hi   target 1  (8 bytes)
x_lo x_hi  y_lo y_hi  v_lo v_hi  r_lo r_hi   target 2
x_lo x_hi  y_lo y_hi  v_lo v_hi  r_lo r_hi   target 3
55 CC                                    tail
```

- `x` , `y` in mm, `v` in cm/s, `r` ("distance resolution", the range gate) in mm.
- **Sign encoding:** bit 15 is the *sign flag* — set means positive, clear means negative — and the lower 15 bits are the magnitude. Ai-Thinker's own worked example: `0E 03` → 0x030E = 782 with the flag clear → **−782 mm**; `B1 86` → 0x86B1, flag set, 0x06B1 → **+1713 mm**; `10 00` → **−16 cm/s**.
- An all-zero 8-byte slot means "no target".
- `+x` is to the radar's **left** (looking out from behind it), `+y` straight ahead. Blip computes distance = √(x²+y²) and bearing = atan2(x, y).
- Speed sign: negative values were observed while approaching. Treat the sign as "as reported by the module" until verified on hardware.

The module also answers commands with ACK frames (`FD FC FB FA … 04 03 02 01`); the parser just resynchronises on the next `AA FF 03 00` and counts the discard in `bad` .

Note: several online sketches decode the frame as `value − 0x200` / `− 0x8000` / `− 0x10` . That matches the worked example only by coincidence for one target; Blip follows the sign-bit rule, which is what the Ai-Thinker protocol note actually describes and what makes the second captured target in `tests/test_parser.cpp` land at a sensible (+483, +2099) mm.

JSON on the wire

Targets (WebSocket text frame or `GET /api/targets`):

```
{"ms":123456,"alive":true,"fps":21,"f":"smart","sleep":false,"t":[
  {"id":7,"x":-782,"y":1713,"v":-16,"res":0,"d":1883,"a":-24.5}],
 "h":[{"id":9,"x":-1100,"y":3000,"why":"ghost"}]}
```

`f` is the target filter. With `raw`, `t` is the module's three slots as they come (`id` 1–3 = slot number). With `smart`, `t` holds Blip's own confirmed tracks: `id` is a track number that stays with a person, and `res` is 0. `h` lists what the Smart filter is hiding and why (`ghost` = likely reflection, `clutter` = wobbling in place); the phone draws these as dashed rings when *Show filtered* is on.

`d` is mm, `a` is degrees (+ left). `alive` turns false 1.5 s after the last good frame.

Stetho (WebSocket binary frames, little-endian, 6-byte header):

Byte	'U' audio	'V' chest motion
0	`0x55`	`0x56`
1	flags: bit0 bump being ducked, bit1 output clipped, bit2 ADC at a rail	same
2–3	sequence number (uint16)	sequence number
4	AGC gain, dB (int8)	0
5	noise suppression, average dB (int8, ≤ 0)	0
6…	256 × G.711 mu-law byte at 8000 Hz (32 ms), **after** the node's clean-up	10 × int16 at 100 Hz (100 ms), raw (DC removed, 80:1 averaged)

The page also still plays the older `'A'` packets (`0x41`, int16 samples). The node's status JSON and target JSON go over the same WebSocket as text; all of it is sent from one task, text first.

The node's clean-up chain (`firmware/include/audio_dsp.h`) is bump gate → preset EQ → 4-band noise suppressor → AGC → limiter. The design is in DESIGN_NOTES.md §11.

The phone: - **Sound:** a ring-buffer player keeps about 40 ms queued, follows the node's clock and skips ahead if it ever lags. It uses an AudioWorklet on https/localhost and a ScriptProcessor on the node's plain-http page. Add `?player=sp` to the URL to force the fallback. - **Chest motion:** 0.1–5 Hz for the drawn trace and the breathing estimate (zero-crossings over 8 s), 0.8–3 Hz for the heart rate (first autocorrelation peak between 0.33 and 1.5 s lag). Samples flagged as bumps are held instead of drawn as spikes. - The node also plays the same processed audio to 3.5 mm earphones (I2S PDM delta-sigma) or an I2S amp board, with or without a phone (hardware/audio-out.md).

Smart target filter (experimental)

`firmware/include/tracker.h`, on top of the module's output, ~20 times a second:

1. **Gate:** drop detections closer than 10 cm, further than 8 m or outside ±65°.
2. **Predict** every track forward with a constant-velocity model.
3. **Associate:** try every pairing of the (≤ 3) detections with the (≤ 6) tracks and keep the cheapest. Distance counts against the gate (0.7 m plus how far a 3.5 m/s walker could go since the track was last seen), and a new track costs more than any match.
4. **Alpha-beta filter** (α 0.5, β 0.15) to smooth position and estimate velocity.
5. **Confirm** a new track only once it has been seen in 3 of its last 5 frames. One-frame flickers never show.
6. **Coast** through missed frames for up to 1 s, then drop the track.
7. **Ghost test:** a track that sits ≥ 0.5 m *behind* a moving track, within 10° of the same bearing, with the module's Doppler speed matching it, for 1 s → hidden as a likely wall reflection. A track that might be one is held back from showing while the test runs.
8. **Clutter map:** a track that keeps being reported but whose ~1 s average position stays within 0.4 m of one spot for 30 s (fan, curtain, swaying plant) is muted. The spot is remembered as a 1.5 m patch of a 0.5 m grid that fades over ~8 minutes; a track sitting still there is re-muted after 3 s instead of 30, and anything that starts up on that spot again is tracked but stays muted. Either comes back if it moves 0.8 m away.

The OLED, the tracker ping and the Scope all use the filtered targets when Smart is on. `tools/tracker_py.py` is a Python copy used by the simulator; `tests/test_tracker_py.py` checks the two agree frame by frame.

Auto-sleep

With *Auto-sleep* set to N minutes: - if no phone is connected and the OLED is off for N minutes, the RD-03D is powered down (needs `PIN_RADAR_PWR`); - if no phone is connected and no node audio output is chosen for N minutes, Stetho stops and the CDM324 is powered down (needs `PIN_STETHO_PWR`; without it only the processing stops).

A phone joining the hotspot wakes both. When the CDM324 comes back on, the first 0.6 s of audio is muted to hide the switch-on thump.

Captive portal

The node answers every DNS query with 192.168.4.1 and redirects the usual connectivity-check URLs (`/generate_204`, `/hotspot-detect.html`, `/connecttest.txt` …) to `/`. Most phones then pop the page up by themselves.

The node log

`firmware/include/blip_log.h` keeps the last ~4 KB of timestamped lines in RTC memory (`RTC_NOINIT_ATTR`), which survives a crash, a watchdog reset or `ESP.restart()`, but not a power cut. Every line also goes to the USB serial port. Every 10 s `loop()` adds a health line:

```
[03:20.0] health: 96 KB free (lowest 81, biggest block 52) | 1 on hotspot, 1 page(s) | +3 requests +12 probes | loop 14 ms | Stetho on 1650
mV, +312 sent +0 skipped +0 dropped
```

To read it: Settings → *Node log* on the page (`/api/log`), or the flasher's 🔌 **USB log** button, which shows the serial output without flashing. Free memory that only goes down across health lines is a leak. A lot of *probes* means a phone or laptop keeps checking for internet (captive portal). Wi-Fi *left, reason N* lines say why a device dropped.

The flasher

The launcher's first page is the **Blip Flasher** (`/` , red): `tools/flasher.py` takes `web/flash_home.html` and inlines `web/flasher.html` with a fresh per-run token. Only this computer gets it; any other client asking for `/` gets the simulator. The simulator, the same page the ESP32 serves, is at `/sim` and shows a small ⚡ **Flasher** button back when the status says `"flash": true` . Neither flasher file is in the firmware, so the board's embedded page stays the same size. `--no-flash` turns all of it off and `/` is the simulator again.

```
browser ──/api/flash/info──────────► boards, radars, serial ports, is PlatformIO there?
        ──/api/flash/start (POST)► {board, radar, port, monitor}   needs X-Blip-Token
        ──/api/flash/log?from=N──► state, 5 step statuses, upload %, tips, new log lines
        ──/api/flash/cancel (POST)
Quick flash:       check firmware/prebuilt/<image> against manifest.json → find/install esptool (.venv)
                   → esptool --chip <chip> --port <port> write_flash 0x0 <image> → serial_tail.py
Build from source: embed_web.py → find/install PlatformIO (.venv) → pio run -e <env> → -t upload → serial_tail.py
```

- **Bundled firmware:** `tools/make_prebuilt.py` copies each env's `firmware.factory.bin` (bootloader + partition table + app) to `firmware/prebuilt/blip-<env>-full.bin` and writes `manifest.json` : version, build time, each image's SHA-256 and size, and a SHA-256 over the firmware source (`firmware/src` , `firmware/include` except the generated `web_ui.h` , `platformio.ini` , `web/index.html` , line endings normalised). Quick flash refuses an image whose checksum doesn't match, and warns (but still flashes the bundle) when your source hash differs.
- **Auto-detect:** Quick flash uses the one USB serial port it can see (ttyACM/ttyUSB, cu.usb*, or the COM ports Windows lists); with none or several it stops and asks, rather than let esptool probe every serial port on the machine.
- **Only this computer:** requests must come from a loopback address with a `localhost` / `127.0.0.1` Host header (stops DNS rebinding); cross-site requests are refused; start and cancel need a random token that is generated per run and only put into the page.
- **No shell:** commands are argument lists. The board and radar pick an env from a fixed table; the port must be one the flasher itself found.
- **Progress:** esptool's "Writing at … (N %)" lines drive the bar for the big segment (the app); the small ones (bootloader, partitions) show as a busy bar. The log keeps one progress line per 10 %.
- **Tips:** each output line is matched against a short list of known problems (port permission, board not in download mode, no internet, Git or `python3-venv` missing, port busy, disk full).
- **Serial output:** `tools/serial_tail.py` opens the port with DTR/RTS low (no reset), retrying for a few seconds while the C3 re-appears on USB.
- **Tests:** `tests/test_flasher.py` runs the whole thing against `tests/fake_pio.py` , which prints what PlatformIO and esptool print and can fail the usual ways.

The simulator

`tools/sim_radar.py` is a stand-in for the whole node: the same URLs, the same JSON, the same WebSocket binary frames (its Stetho sound is synthesised to look plausible, not run through the real DSP), Python standard library only. Its three walkers are **encoded into real 30-byte RD-03D frames and decoded again** by a Python port of the parser before they reach the page, so the byte layout is exercised every time you run it. `--frames examples/frames_synthetic.hex` replays a hex dump instead.

CHAPTER 3

Design notes and research

DESIGN_NOTES.md

Written 2026-11-05 while building v0.1.0-wip to v0.2.2-wip. Part lab notebook, part "why", with the sources used. Nothing here has been checked against real hardware yet unless it says so.

1. Goal

Rebuild the idea behind Stevee87's Arduino-ESP32-Radarproject with the least hardware that still does the job: one cheap ESP32 (an **ESP32-C3 Super Mini** or a **Waveshare ESP32-S3-Pico**), the same **Ai-Thinker RD-03D** radar, and an **old phone** on the node's own hotspot as the display. Add the generic **0.91"/0.96" SSD1306 OLEDs** as a second display. Then look seriously at the "Doppler stethoscope" wish: can this radar let you *hear hearts through walls*?

2. What the original does, and why it felt heavy

Stevee87's build: XIAO ESP32-S3 reads the radar and sends target data over UDP to an **Arduino GIGA R1 + GIGA Display Shield**, which draws the scope and drives a buzzer. Two LiPos, two chargers, two boost converters. The second, much more expensive MCU exists only to own a touchscreen. A phone has a better screen, touch, a speaker and a battery, and every household has an old one in a drawer. So: the ESP32 serves a web page, the phone draws.

Original repo, MIT: https://github.com/Stevee87/Arduino-ESP32-Radarproject — kept as inspiration; no code copied.

3. The RD-03D, what it really is

From the Ai-Thinker Rd-03D specification V1.0.0 (2023) and the protocol note:

Item	Value
Chip	**Iclegend S5KM312CL** — "integrated microcontroller millimetre-wave sensor SoC", FMCW transceiver + MCU in one package
Band	24.0–24.25 GHz (K-band ISM), max sweep bandwidth 0.25 GHz
Antennas	**1 TX, 2 RX** on the PCB ("1T2R")
Range	8 m max, range resolution 0.75 m, accuracy 0.15 m
Field of view	±60° azimuth, ±30° elevation
Supply	5 V, ≥200 mA peak, ~92 mA typical, ripple < 100 mV
Interface	UART, 3.3 V logic, default 256000 baud
Pins on the connector	**4: 5V, GND, TX, RX** — that's all
Output	up to 3 tracked targets: x, y (mm), radial speed (cm/s), range gate (mm)

About "multi-channel / multi-band": the module is **one band** (24 GHz) with **two receive channels**. The two RX antennas let the chip's firmware estimate bearing from the phase difference between them (that's where the x coordinate comes from). It is not a multi-band radar, and the two channels are not accessible from outside the chip.

The datasheet's "GPIO" section mentions that "some IO ports are drawn from the periphery of the module" and that they are 3.3 V. These are unlabelled test pads; nothing documents what they are. There is no public SDK for the S5KM312CL that I could find (searches for an Iclegend SDK / raw ADC / I-Q output came up empty), and the chip's firmware is Ai-Thinker's. **The range-Doppler processing, target detection and tracking all happen inside the chip; only the tracker's answers come out.**

3.1 Frame format and the sign-bit gotcha

Covered in HOW_IT_WORKS.md. The short version: each of x, y, v is a 15-bit magnitude with bit 15 = "positive". Ai-Thinker's worked example (`0E 03` → −782 mm, `B1 86` → +1713 mm, `10 00` → −16 cm/s) is in both test suites. Popular sketches that subtract 0x200/0x8000/0x10 only happen to agree with that one example; Blip uses the sign bit.

3.2 Report rate

Users report the module streams "very frequently" (ESPHome's component adds a throttle for that reason). I have not measured it; the firmware counts frames per second and shows it in the UI so the first real run will tell us. The web UI is rate-limited to 20 updates/s regardless.

4. Could the RD-03D be bit-banged into a Doppler stethoscope?

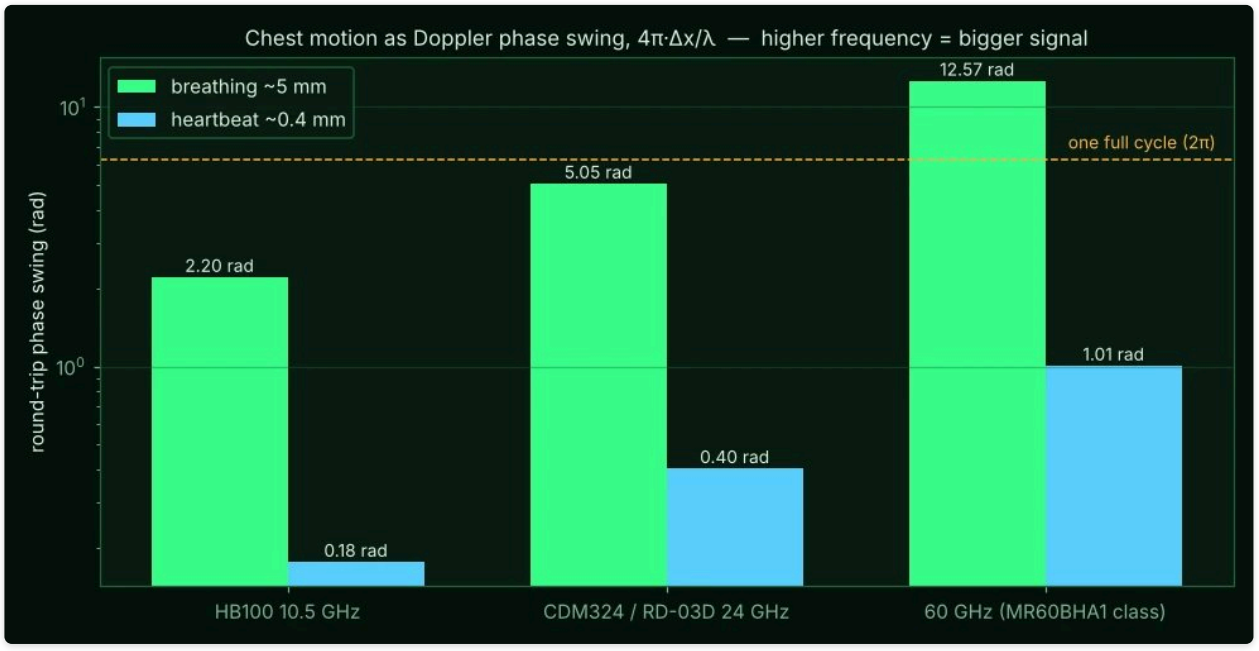
Short answer: **no, not through its UART**, and probably not at all without reverse-engineering an undocumented SoC. Long answer, with numbers.

4.1 What a heartbeat looks like to a radar

A CW or FMCW radar sees a target's motion as the phase of the reflected signal. Moving the chest wall by Δx changes the round-trip phase by

> $\Delta\varphi = 4\pi \cdot \Delta x / \lambda$

- Breathing moves the chest ~4–12 mm; heartbeat moves the skin surface ~0.2–0.5 mm.
- At 24 GHz, λ = 12.5 mm → breathing ≈ 5 rad (nearly a full cycle), heartbeat ≈ **0.4 rad** (≈ 23°).
- At 10.5 GHz (HB100), λ = 28.5 mm → heartbeat ≈ 0.18 rad.
- At 60 GHz the heartbeat swing is ~1 rad, which is why the commercial vital-sign modules (Seeed MR60BHA1 and friends) sit at 60 GHz.



To extract a heartbeat you need the **I/Q baseband** (or at least one channel of it) sampled at tens of Hz with sub-tenth-of-a-radian stability, then a band-pass around 0.8–3 Hz and some patience: the breathing signal is ten times bigger and sits next door in frequency.

4.2 What the RD-03D gives you instead

- Radial **speed as an integer in cm/s** for each *tracked* target. A heartbeat's chest velocity is of order 0.4 mm × 2π × 1.2 Hz ≈ 3 mm/s = **0.3 cm/s** — below the resolution of the number, and the tracker is built to ignore exactly this kind of micro-motion as "static".
- Position in **0.75 m range gates**, filtered by a tracker that holds targets for a configurable retention time.
- A chip whose internal ADC, mixer outputs and FFT buffers are not on any pin.

So the information needed for a stethoscope is destroyed inside the chip long before the UART. "Bit-banging" would mean finding the chip's debug port (SWD-like pads on the module edge are plausible but undocumented), dumping and replacing Ai-Thinker's firmware, and writing a radar signal chain for an SoC with no public datasheet. That's a research project in its own right; it is parked, not forgotten (see §9).

4.3 What *does* work: a CW Doppler module with an IF pin

The classic motion-sensor modules put the mixer output on a pin:

Module	Band	IF output	Notes
HB100	10.525 GHz	yes, µV–mV	the £3 standard; through-wall capable; 70 Hz per m/s
CDM324	24.125 GHz	yes (IF pin)	same footprint as HB100, 24 GHz so 2.3× the phase swing; 160 Hz per m/s

Either one, plus an amplifier into the ESP32 ADC, gives a waveform that *is* the chest (or throat, or walker) motion. That is the **Stetho channel** in Blip. v0.1 sent the raw 2 kHz waveform to the phone and did everything there; v0.2 moves the clean-up onto the ESP32 and uses a modded MAX4466 microphone board as the amplifier (§11).

4.4 "Hearing" a heart

There are two different things to hear:

- **The beat itself** is ~1 Hz chest *motion*, far below hearing. It can only be shown (the chest-motion trace) or translated (a synthesised "thump" on each detected beat — the *Beat thumps* button).
- **Heart sounds** (S1/S2, the valves closing) are genuine vibrations of the chest wall at roughly 20–150 Hz, and a CW radar can pick them up: Will et al. (2018, Scientific Reports) detected S1 and S2 contact-free with a CW radar and matched a phonocardiograph's timing closely (§Sources). That's what the **Heart** preset tries to let you hear directly. The throat buzzes at the voice's fundamental (85–255 Hz) while someone talks (**Voice** preset), and breath noise is broadband hiss (**Lungs**). People walking make the Doppler warble (**Raw**), and that does go through plasterboard.

4.5 Through walls: what the literature says

- Pramanik & Islam, *Through the wall human heart beat detection using single channel CW radar*, Frontiers in Physiology 2024: a single-channel 24 GHz CW radar recovered heart rate through wood and **brick** barriers for subjects **20–100 cm** from the radar, with wavelet processing (MODWT), and even separated two people. https://www.frontiersin.org/journals/physiology/articles/10.3389/fphys.2024.1344221
- Georgia Tech's "Radar Flashlight" (1998): 24.1 GHz, breathing detected through doors and walls at up to ~10 ft; heartbeat at longer range only with a much larger antenna. https://www.designnews.com/industry/hand-held-radar-device-detects-breathing-heartbeats
- Malešević et al., *Contactless Real-Time Heartbeat Detection via 24 GHz Continuous-Wave Doppler Radar Using Artificial Neural Networks*, Sensors 2020: heartbeat-by-heartbeat detection at 24 GHz CW, line of sight. https://pmc.ncbi.nlm.nih.gov/articles/PMC7219229

Realistic expectations for a £3 module and a breadboard amplifier:

Target	Line of sight	Through a plasterboard stud wall	Through brick
Walking person	easy, many metres	yes, reduced range	maybe, close
Breathing, person sitting still	yes at 1–2 m	plausible at < 1 m	doubtful
Heartbeat	plausible at < 1 m, subject very still, breath held helps	**hard**; the paper managed ≤ 1 m with careful processing	no

Everything else in the room must be still: a curtain moving is a bigger signal than a heart. Expect the first real test to be "breathing through a door", which is still a great party trick.

Not a medical device, and treat the privacy angle with care: this is the same physics as the search-and-rescue life detectors, in a hobby box.

5. Decisions and the alternatives dropped

Decision	Why	Rejected
Node is its own Wi-Fi **hotspot**	works in a field, in a basement, with any old phone; no router config	STA mode joining home Wi-Fi (maybe later as an option), BLE (no browser UI), ESP-NOW to a second node (that's the two-MCU design again)
WebSocket for data, **HTTP polling** as fallback	20 Hz updates without reconnect overhead; polling keeps ancient browsers working	Server-Sent Events (no binary)
ESPAsyncWebServer (ESP32Async fork)	non-blocking, WebSocket built in, still maintained in 2025	Arduino `WebServer` (no WebSocket; blocks), ESP-IDF httpd (not Arduino-friendly)
Single HTML file **gzipped into flash**	no filesystem image to upload separately; `tools/embed_web.py` rebuilds it	LittleFS data upload (one more step for the flasher)
U8g2 for the OLED, runtime-selectable size	one library covers 128×32 and 128×64; the size is a setting, not a build flag	Adafruit_SSD1306 (needs the size at compile time)
Raw Stetho samples to the phone	DSP in JS is quicker to iterate; phone has the speaker	on-node FFT/rate estimation
Pins from `platformio.ini` only	diagrams and `/api/status` read the same numbers; nothing to drift	pins in `config.h` and in docs separately
C3 Super Mini pins: radar on GPIO20/21, I2C on GPIO6/7, ADC on GPIO1	20/21 are the board's RX/TX pads (UART0 is free because the C3 uses native USB-CDC); 6/7 are plain IO (JTAG, unused); GPIO1 is ADC1. Avoided: GPIO5 (ADC2 is unusable with Wi-Fi), GPIO2/8/9 (strapping pins, LED, BOOT)	—
S3-Pico pins: radar on GPIO12/11, I2C on GPIO17/18, ADC on GPIO7	these are the Pico-position GP0/GP1, GP6/GP7 and A1 that Waveshare's own UART/I2C/ADC demos use, so a Pico breadboard layout carries over; GPIO7 is ADC1	—
PolyForm Noncommercial licence	the project's own code; nothing GPL incorporated	—

6. Measurements

None yet on hardware. From the simulator and tests:

- Parser: 3 frames recovered from a stream containing junk, a 12-byte ACK and a half frame; header-like bytes inside the payload do not break framing (`tests/test_parser.cpp` , `tests/test_parser_py.py`).
- Web UI: 25.5 kB HTML → 9.7 kB gzip in flash.
- Stetho estimator on the synthetic signal (72 bpm beats on 15/min breathing with noise): reported 73 bpm and 15/min after ~8 s (Chromium, Playwright run).
- Simulator → Chromium: WebSocket delivered 79 messages in 2 s (40 target frames + 39 binary Stetho frames) with zero console errors.

7. What was tried and dropped

- Hijacking the HTTP socket for WebSocket in the Python simulator initially failed silently because `socketserver` shuts the socket down after the handler returns; fixed by overriding `shutdown_request` for hand-offs (`tools/sim_radar.py`).
- Heart-rate estimate picked the autocorrelation's global maximum, which was the shortest allowed lag whenever breathing leaked in (150 bpm). Switched to a steeper 0.8 Hz high-pass (two cascaded biquads) and the *first local* peak of the autocorrelation.

- A `display:flex` modal ignored the `hidden` attribute (`[hidden]{display:none!important}` fixed it).

8. Untested / unknown

- **Firmware compiles but has not run.** v0.2.0-wip builds cleanly for both envs with pioarduino (Arduino-ESP32 3.3.12). C3: 13% RAM, app 1.18 MB in the 3 MB `huge_app` partition. S3: 16% RAM. Nothing has been flashed yet.
- ADC continuous (DMA) mode at 8 kHz alongside Wi-Fi on the single-core C3: should be fine (it's DMA), but unmeasured. `overruns` and `dropped` in `/api/status` will tell.
- I2S PDM-TX at 8 kHz on the C3/S3 is used for the earphone output; the clock/up-sampling defaults come from Arduino's `ESP_I2S` . Untested by ear. The ADC and the I2S run from different clock dividers, so a tiny drift may cause a click every few minutes.
- The real level out of a CDM324 + MAX4466 at 0.3–1 m, and therefore whether heart sounds are audible at all, is unknown until someone builds it.
- The DSP thresholds (bump trigger, noise floor tracking, AGC targets) are tuned on a synthetic scene only.
- Sign convention of the reported speed (approaching = negative is assumed from one write-up).
- Actual RD-03D frame rate.
- Whether the captive portal pops on iOS as well as Android.
- U8g2 HW-I2C constructor with explicit SCL/SDA pins on the C3.
- The MAX4466 mod is reasoned from the board's topology, not measured; the component values on clone boards vary.
- The S3-Pico GP2–GP5 = GPIO13–16 mapping is inferred from Waveshare's sequential numbering; check before soldering.

9. Ideas for later

- **I/Q Stetho:** a second ADC1 pin would do it (v0.2 dropped the reserved `PIN_STETHO_Q` to free pins for audio out). With a quadrature module (or two HB100s offset by λ/8) the phase can be unwrapped properly and the "null point" problem of single-channel CW goes away.
- Use the RD-03D's "someone is at 0.8 m, dead ahead" to **gate** the Stetho channel (only sonify when a target is close and still).
- CSV logging of the Stetho waveform in the browser for offline analysis; a spectrogram view.
- STA mode + mDNS (`blip.local`) as an option.
- Poke at the S5KM312CL's edge pads with a logic analyser during boot: if there is a UART debug console or SWD, note it here.
- Enclosure with a 3.7 V cell + boost, like the original.

10. How to test a change

1. `g++ -std=c++11 -I firmware/include tests/test_parser.cpp -o /tmp/t && /tmp/t` and `python3 tests/test_parser_py.py` — both print `RESULT: PASS` . `g++ -std=c++17 -I firmware/include tests/test_audio_dsp.cpp -o /tmp/d && /tmp/d` prints `ALL PASS` .
2. `python3 tools/embed_web.py` after touching `web/index.html` .
3. `python3 tools/sim_radar.py --no-browser` and open the page on a phone; check the scope, Settings round-trips, and that Stetho shows ~72 bpm / 15 per min and plays sound after *Listen on phone*.
4. `python3 tools/draw_illustrations.py` if a pin changed; look at every image in `docs/images/` .
5. On hardware: `pio run -e c3-supermini -t upload` , watch the serial monitor for `Blip 0.2.0-wip` , join the hotspot, and compare `/api/status` counters (`frames` rising, `bad` near zero).

11. v0.2 — the audio build (MAX4466, on-node DSP, three outputs)

Asked for: use a MAX4466 board instead of hand-biased op-amps; push live audio to the phone over the WebSocket with as little delay as possible; add delta-sigma audio for 3.5 mm earphones and a cheap amp board; do the AGC and noise suppression on the ESP32, tuned to reject bumps and lift heart, lungs and larynx; warn about DC on the old mic pads; and say whether the RD-03D and a CDM324 clash.

11.1 MAX4466 board as the amplifier

The board is a complete single-supply preamp: mid-rail bias, adjustable gain (×25–×125), output idling at VCC/2. Its gain leg is AC-coupled (DC gain ≈ 1), so a radar IF offset can't rail it. It is an *audio* amp, so its low-frequency corner (tens of Hz, depending on the board) is right for heart sounds and voice, and weak for the 0.2–2 Hz chest motion. Upsizing the gain-leg cap fixes that (hardware/max4466-mod.md). The important trap is the **mic bias resistor**, which would push ~1.5 mA of DC into the CDM324's IF pin, so the mod removes it (or adds a series DC-block cap). ×125 is less gain than the old two-stage plan (×3000+). Two modded boards in series cover the gap, and the on-node AGC adds up to 48 dB digitally. Its limit is the 12-bit ADC's own noise.

11.2 Why the DSP moved onto the ESP32

Earphones and the amp board have to work with no phone. So the node does the clean-up, and the phone, earphones and speaker all hear the same processed signal. The C3 has **no FPU**, so the per-sample path is fixed-point: Q28 biquads with 8 extra state bits **and error feedback**. Without it, the native test showed a 3 Hz high-pass at 8 kHz sitting ~350 LSB off zero from truncation bias, the 1/(1+a1+a2) gain at work. An earlier version of the bump detector only "passed" its test because of that offset; fixing the filter exposed it, and the detector was redesigned (below). The estimated cost is about 500 cycles a sample, i.e. a few % of the C3 at 8 kHz.

Chain (`firmware/include/audio_dsp.h` , tested by `tests/test_audio_dsp.cpp`):

1. **Bump gate.** A fast envelope (0.5 ms attack) of the near-raw signal is compared with a background of *typical peaks* (0.3 s up, 3 s down, frozen during a bump). Above k× the background (k = 30…3 from the *Bump rejection* slider), or with the ADC at a rail, the audio is ducked by 30 dB within 1 ms, held for 150 ms and released over 120 ms. A 4 ms look-ahead delay means the duck lands *on* the knock's leading edge, not after it. Tracking peaks rather than the mean is what keeps heart sounds (spiky, ~10% duty) from triggering it.

2. **Preset EQ.** High-pass, one peaking lift, low-pass. Heart: 15–250 Hz, +6 dB at 45 Hz. Lungs: 100–1500 Hz, +4 dB at 400 Hz. Voice/larynx: 70–3000 Hz, +4 dB at 200 Hz. Wide: 15–3400 Hz. Raw: 3–3600 Hz, nothing else.
3. **Noise suppressor**, the same idea as SquawkBox's goal of "remove the background, keep the voice". (Only my notes on SquawkBox were reachable from here, not its code, so this is a fresh implementation of the usual approach.) Four band-passes split the signal. Each band tracks its own noise floor (falls fast to quiet stretches, rises over ~6 s) and gets a Wiener-style gain g = 1 − α·noise/envelope, clamped to a floor (−6…−24 dB from the slider). The output is `g_res·(x − Σbands) + Σ g_i·band_i`, so with every gain at 1 it is *exactly* the input: no colouring when there's nothing to remove.
4. **AGC.** It follows a peak-ish level (5 ms up, 1 s down) towards about −9 dBFS. It is frozen during bumps, so a knock doesn't pump the gain down. It is capped so the noise floor never rises above about −24 dBFS, so silence stays quiet rather than turning into hiss. The ceiling is user-set (0–48 dB), and there is a manual mode.
5. **Limiter.** Soft knee at 24000, hard clip flagged to the phone.

The native test result on the synthetic scene (heart sounds + noise + knocks 25 dB louder): knocks end up **~38 dB** down relative to heart sounds, the hiss between beats **~11 dB** down, no false bump triggers over 45 heart-sound bursts, and no clipping. The Raw preset passes a tone at 0.00 dB gain with 66 dB SNR. `examples/dsp_before.wav` and `dsp_after.wav` are that scene, so you can listen; `docs/images/dsp-demo.webp` shows it.



11.3 "Unbuffered, no latency" to the phone — what's actually possible

Truly unbuffered audio over Wi-Fi isn't possible: packets arrive in bursts and a player with no buffer clicks constantly. Instead, every stage is kept as short as it can be: - The ADC runs in DMA mode, and the node processes in 8 ms blocks. - The node sends 256-sample (32 ms) packets of mu-law bytes with `TCP_NODELAY` and Wi-Fi power-save off, from `loop()`. A test build sent them from a separate FreeRTOS task instead, and the C3 froze within seconds of Stetho starting (no reset, web server dead). It is most likely a lock-up inside AsyncTCP/ESPAsyncWebServer when two tasks send at once, but that isn't confirmed, so all WebSocket output stays in `loop()`. That is about 33 audio messages a second at ~8 KB/s. The first version sent 128 int16 samples (16 ms) from `loop()`. On a C3 talking to a laptop, about 90 small messages a second (audio + vitals + targets + status) overran what the link could acknowledge: packets were dropped, audio lagged by seconds and the status stopped updating. Each phone's queue is now capped at 8 messages, and full queues discard, so playback never drifts behind real time. The phone's *Delay* readout counts the losses. - The phone plays through a ring buffer that aims at **40 ms**, nudges its playback rate ±2% to follow the node's clock, and jumps forward if it falls 120 ms behind. It runs in an AudioWorklet where the browser allows it (https/localhost) and otherwise in a ScriptProcessor (512 frames). The node's page is plain http, so on a real phone it will be the ScriptProcessor. - In the simulator, Chromium reported 54–83 ms end to end (buffer + output latency + one packet). Expect roughly 60–120 ms on an old phone. The earphone and speaker outputs on the node skip Wi-Fi entirely: about 8 ms block + I2S DMA, i.e. ~20 ms.

11.4 Delta-sigma earphones, I2S amp

The "pseudo-PWM" output uses the I2S peripheral in **PDM-TX mode**, which is a hardware delta-sigma modulator clocked by DMA (≈2 MHz bit stream at 8 kHz audio). The SDM peripheral (`sigmaDeltaWrite`) would also work, but it has to be fed a new value every 125 µs by software, which means timer jitter on a single-core chip that's also running Wi-Fi. PDM needs only one data pin, plus a 470 Ω / 100 nF RC and a 220 µF coupling cap. The amp option is a MAX98357A (I2S, despite "I2C" often being written). Both share the one I2S peripheral on the C3, so it's one output at a time.

11.5 Running the RD-03D and a CDM324 together

Yes, they can clash. The RD-03D is FMCW in 24.00–24.25 GHz (§3). The CDM324 is a fixed CW tone at ~24.125 GHz, inside that sweep. Each time the RD-03D's chirp sweeps past 24.125 GHz, the CDM324's mixer sees a beat note racing through 0 Hz, which is a click. At the chirp/frame rate those clicks become a buzz in exactly the band you are listening to. Going the other way, the CDM324's steady tone shows up in the RD-03D's dechirped signal as an interference burst, which can raise its noise floor or add ghosts. How bad depends on how much one antenna sees of the other (side by side, facing the same way: some; facing each other: a lot). It is untested.

Options, best first: 1. **Use an HB100 (10.525 GHz) for the Stetho.** It is a different band, so there is no clash. The cost is ~2.3× less phase swing (weaker heart signal). 2. **Switch the radar off while listening.** v0.2 supports an optional high-side switch on the RD-03D's 5 V (`PIN_RADAR_PWR` , Settings → *Radar off*). Tracking stops while Stetho is on. 3. Physical: mount them back to back, or with metal or absorber between them. The bump gate will catch some clicks but not a steady buzz.

12. v0.2.1 — a better target filter, the mode-command fix, power and the OLED

12.1 What the original project actually does with the targets

I read Stevee87's firmware (v1.1). The "tracking algorithm" in its README is the RD-03D's own, inside the chip. On top of it, his sketches add three simple things: 1. **Range gate:** keep only targets between 10 cm and 8 m. 2. **Clustering** (sender): two targets closer than 1 m are merged into one (averaged position, faster speed kept), aimed at "torso + waving arm = two people". 3. **Still-ghost filter** (receiver): a slot whose position hasn't changed by more than a few cm for 2 s is hidden, on the reasoning that the RD-03D doesn't hold still people anyway, so a dot that doesn't move is suspicious.

His frame decoding uses the same sign-bit rule as Blip (`raw ≥ 0x8000 → raw − 0x8000` , else `−raw`), which confirms §3.1.

It also showed a bug in Blip: his mode change is *enable configuration* → *multi-target* → *end configuration*, 200 ms apart. Blip ≤ v0.2.0 sent only the middle command, which the module very likely ignored, so the single/multi setting probably did nothing. Fixed in v0.2.1 with a non-blocking 3-step sequence run from `loop()` .

12.2 Why not just copy those filters

- **Merging within 1 m** also merges two real people walking close together, and it works on the chip's slot numbers, which can swap between frames.
- **Hiding still slots** works per slot, so a slot swap moves the "still" timer to the wrong person. It also hides a person the moment they stop, which is usually when you want to see where they are.
- **Neither tackles the two big sources of false positives:** one-frame flickers and multipath reflections. A reflection is the radar seeing you via a wall or window, and it appears as a second "person" further out, moving in step with you.

12.3 What bigger mmWave people trackers do

TI's people-counting demo ("group tracker") and MathWorks' mmWave people-tracking example share a common structure: **gating → association → per-track filter (Kalman / alpha-beta) → track management** (tentative tracks confirmed after M-of-N detections, deleted after misses), plus clutter handling (static/"boundary" regions, minimum points and SNR to allocate a track). The RD-03D gives no SNR or point clouds, only up to 3 already-clustered points, so Blip uses the same structure scaled down: brute-force association (≤ 3 × 6 pairings), alpha-beta filters, 3-of-5 confirmation and 1 s coasting. On top of that are two cheap heuristics for what the chip gets wrong most: the same-bearing/lockstep **ghost test** and a learned **clutter map**. The ghost test compares the module's own (smoothed) Doppler speeds rather than speeds worked out from positions, because the latter came out far too noisy in the first test run (the ghost was never caught). Details in HOW_IT_WORKS.md.

12.4 How it was tested (synthetic, `tests/test_tracker.cpp`)

At 20 Hz with ~120 mm position noise and the chip's slots shuffled every frame:

Scene	Result
one walker, 60 s, 0.3 s dropouts	kept **one id** throughout; position error 93 mm rms vs 126 mm raw
15% one-frame flickers	**0** shown in 1159 frames
wall reflection 1.2 m behind (seen 80% of frames)	hidden **100%** of the time; the real person shown 100%
fan wobbling in place	muted after 30 s (0 frames shown after 35 s); a person walking past it still shown 100%
two people crossing 0.6 m apart	both kept their own ids
real person behind, moving differently	never mistaken for a ghost
detections at 79°, 5 cm, 9 m	never shown
known limitation: two people in single file, in step	the second is hidden (it looks exactly like a reflection)

In the simulator's messy room (two walkers, a wall reflection, a fan and flickers; 4 random seeds × 150 s), once the fan had been learned it was shown in 0–6% of frames when nobody was near it, and a walker was shown in 92–99% of frames. The misses are mostly the chip's 3-slot limit, with the reflection and the fan taking slots. Two fixes came out of that run: stillness is now judged on a ~1 s average position, because noise and coasting kept resetting the timer, and a learned spot covers a 1.5 m patch, because the fan sat on a cell edge.

`tests/test_tracker_py.py` replays 3 minutes of messy detections through the C++ and the simulator's Python copy: the same ids in 99.94% of frames, positions within 1 mm.

That's why it is an **opt-in experimental mode** (Settings → Target filter → Smart), with *Show filtered* on the Scope so you can see what it hides. Real rooms will need tuning; the thresholds are in `blip_track::Cfg` .

12.5 Power: no "sleep" on either radar

I found no documented sleep or transmitter-off command for the RD-03D. Its config mode stops the UART reports, but nothing says the 24 GHz sweep stops. The CDM324/HB100 are an oscillator and a mixer with no enable pin. So power is cut with optional high-side switches (`PIN_RADAR_PWR` , `PIN_STETHO_PWR`). Don't switch ground: the RD-03D's UART lines would back-feed it, and a floating CDM324 ground puts a big step through the MAX4466's coupling cap. *Auto-sleep* powers each radar down after N minutes with nobody watching or listening.

12.6 OLED

v0.2.0 drew a dot plus a slot number and spent a whole text row on "3 tgt 1.88m −16cm/s". From v0.2.1: - **Each person is a circle with their distance inside:** whole metres in a 5×7 (or 4×6) digit and tenths as a smaller raised 3×5 digit, so 1.9 m reads "1⁹" with no decimal point. The nearest person is drawn solid (inverted). - **No status row at all.** The 120° sector is stretched to fill the panel. The Smart-filter diamond sits in the top-left corner and the Wi-Fi icon with the phone count in the top-right, both outside the sector. The IP address appears bottom-left only while no phone is connected. "NO RADAR" and "SLEEP" appear in the middle only when that's the case. - The drawing code (`oled_draw.h`) writes into a plain buffer in the SSD1306 page layout. The firmware copies it into U8g2's frame buffer, and `tests/oled_render.cpp` runs the identical code on a PC to make the README previews. They are exact renders, not mock-ups.

Sources

- Stevee87, *Arduino-ESP32-Radarproject* firmware v1.1 (`rd03d_xiao_s3_sender.ino` , `rd03d_giga_receiver.ino`) — range gate, 1 m clustering, 2 s still-ghost filter, enable/mode/end command sequence: https://github.com/Stevee87/Arduino-ESP32-Radarproject
- MathWorks, *People tracking using mmWave radar* (gating, association, track confirmation): https://www.mathworks.com/help/radar/ug/people-tracking-mmwaveradar-example.html
- TI, *3D people counting tracker layer tuning guide* and *pplcount customization guide* (allocation, gating, state transitions, static/boundary handling): https://dev.ti.com/tirex/content/mmwave_industrial_toolbox_4_11_0/labs/People_Counting/docs/3D_people_counting_tracker_layer_tuning_guide.pdf and https://e2e.ti.com/cfs-file/__key/communityserver-discussions-components-files/1023/pplcount_5F00_customization_5F00_guide.pdf
- Ai-Thinker, *Rd-03D Specification V1.0.0* (2023): https://en.ai-thinker.com/Uploads/file/20231016/20231016032622_13559.pdf
- Ai-Thinker Rd-03D product page / Digi-Key listing (S5KM312CL, 256 kbps): https://www.digikey.com/en/products/detail/ai-thinker/RD-03D/24614840
- Electronic Clinic, *Rd-03D mmWave Radar: Multi-Human Tracking...* (2024) — frame examples and commands: https://www.electroniclinic.com/rd-03d-mmwave-radar-multi-human-tracking-with-distance-speed-positioning/
- ESPHome RD-03D component (256000 baud, Doppler-only caveat, throttle): https://esphome.io/components/sensor/rd03d/
- Stevee87, *Arduino-ESP32-Radarproject*: https://github.com/Stevee87/Arduino-ESP32-Radarproject
- Waveshare ESP32-S3-Pico wiki (pin map, demos): https://www.waveshare.com/wiki/ESP32-S3-Pico
- ESP32-C3 Super Mini pin references: https://lastminuteengineers.com/esp32-c3-super-mini-pinout-reference/ and https://www.sudo.is/docs/esphome/boards/esp32c3supermini/
- Pramanik & Islam (2024), Frontiers in Physiology: https://www.frontiersin.org/journals/physiology/articles/10.3389/fphys.2024.1344221
- Malešević et al. (2020), Sensors, PMC7219229: https://pmc.ncbi.nlm.nih.gov/articles/PMC7219229
- Design News (1998), GTRI Radar Flashlight: https://www.designnews.com/industry/hand-held-radar-device-detects-breathing-heartbeats
- Will, Shi, Schellenberger, Steigleder, Michler, Fuchs, Weigel, Ostgathe, Kölpin, *Radar-Based Heart Sound Detection*, Scientific Reports 8:11551 (2018), doi:10.1038/s41598-018-29984-5 — CW radar, S1/S2 detected, F1 ≈ 92% vs ECG: https://cris.fau.de/publications/202373734/
- Analog Devices EngineerZone, MAX4466 application-circuit thread (bias, coupling and corner-frequency discussion): https://ez.analog.com/audio/f/q-a/599838/clarification-regarding-the-max4466-application-circuit
- Core Electronics forum, Adafruit MAX4466 (ADA1063) schematic discussion (AC-coupled gain leg, unity gain at low frequency): https://forum.core-electronics.com.au/t/electret-microphone-amplifier-max4466-ada1063/3122
- Adafruit MAX4466 product page (gain 25×–125×, 2.4–5.5 V): https://www.adafruit.com/product/1063
- limpkin's HB100/CDM324 Doppler backpack (speed-measuring amp, frequency output; retired): https://www.tindie.com/products/limpkin/hb100-doppler-speed-sensor-arduino-compatible/
- ESP-IDF 5.5 headers shipped with Arduino-ESP32 3.3.12: `esp_adc/adc_continuous.h` (`adc_continuous_read_parse`), `ESP_I2S.h` (`I2S_MODE_PDM_TX`), `soc_caps.h` (`SOC_I2S_SUPPORTS_PDM_TX = 1` on C3 and S3).
- HB100 amplifier notes: https://www.instructables.com/Easy-HB100-Amplifier and https://tmrh20.blogspot.com/2016/06/hb-100-gh1420-10.html

CHAPTER 4

Wiring

hardware/wiring.md

See the diagrams in `docs/images/` (drawn from `firmware/platformio.ini` by `tools/draw_illustrations.py`).

Signal	ESP32-C3 Super Mini	Waveshare ESP32-S3-Pico	Note
Radar TX → ESP RX	GPIO20	GPIO12	UART1, 256000 8N1, 3.3 V logic
Radar RX ← ESP TX	GPIO21	GPIO11	mode command only
Radar 5V	5V pin	VBUS	≥200 mA peak, ripple <100 mV; never 3V3
OLED SDA / SCL	GPIO6 / GPIO7	GPIO17 / GPIO18	SSD1306 at 0x3C, powered from 3V3
Stetho: MAX4466 OUT	GPIO1	GPIO7	ADC1; MAX4466 on 3V3 — see max4466-mod.md
Earphones (delta-sigma)	GPIO10	GPIO16	through the RC filter in audio-out.md
I2S amp BCLK / LRC / DIN	GPIO3 / GPIO4 / GPIO5	GPIO13 / GPIO14 / GPIO15	MAX98357A board
Radar power switch (optional)	not assigned	not assigned	set `PIN_RADAR_PWR` to use one
CDM324 power switch (optional)	not assigned	not assigned	set `PIN_STETHO_PWR`; same kind of high-side switch

Notes - The RD-03D connector is a 1.25 mm 4-pin socket (5V, GND, TX, RX). Keep the wires short; the module is ESD sensitive. - The C3 Super Mini's USB-C 5V pin can feed the radar for bench tests. For battery use add a 1S cell + 5 V boost (the radar wants 5 V, the datasheet asks for an LDO or a low-ripple DC-DC). - Keep metal away from the radar's antenna face; a metal plate *behind* it shields the back lobe. - Only ADC1 pins work while Wi-Fi is on (C3: GPIO0–4, S3: GPIO1–10), which is why the Stetho input is where it is. - **Optional radar power switch:** to switch the RD-03D off while a CDM324 listens on the same band, put a high-side switch on its 5 V (a P-MOSFET module or a logic-level load switch). Then set `-DPIN_RADAR_PWR=<gpio>` in `platformio.ini` and pick *Radar off* in Settings. Don't switch its ground: the UART lines would then back-feed the module. - **Optional CDM324 power switch:** the CDM324/HB100 has no enable pin and transmits whenever powered. Put the same kind of high-side switch on its 5 V and set `-DPIN_STETHO_PWR=<gpio>`; it then only runs while Stetho is on, and Auto-sleep can turn it off. Don't run it straight from a GPIO: it wants 5 V at about 35 mA. - How to build the switch: a P-MOSFET (e.g. AO3401) in the 5 V line, its gate pulled up to 5 V with 10 kΩ, and a small N-MOSFET (e.g. 2N7002 or AO3400) from that gate to GND driven by the GPIO. GPIO high = N-FET on = P-FET on = module powered. A 3.3 V pin can't switch a P-FET on 5 V directly. - Free pins: on the C3 use **GPIO0** for one switch; a second has to be **GPIO2**, a strapping pin that must not be pulled low at boot. An N-MOSFET gate draws no current, so that's fine, but don't use a bipolar transistor or a pull-down there. On the S3-Pico, GPIO8 and GPIO9 are free. - **HLK-LD2450 instead of the RD-03D:** same 5 V / GND / UART wiring in the table above. Its header is labelled from the module's side (5V, GND, RX, TX), so its **TX** goes to the board's **RX** pin (GPIO20 on the C3, GPIO12 on the S3-Pico) and its **RX** to the board's **TX** pin. Build with `c3-supermini-ld2450` or `s3-pico-ld2450`. - The firmware compiles for both boards (v0.2.2-wip), with the RD-03D and with the LD2450 option. None of this has been verified on real hardware yet.

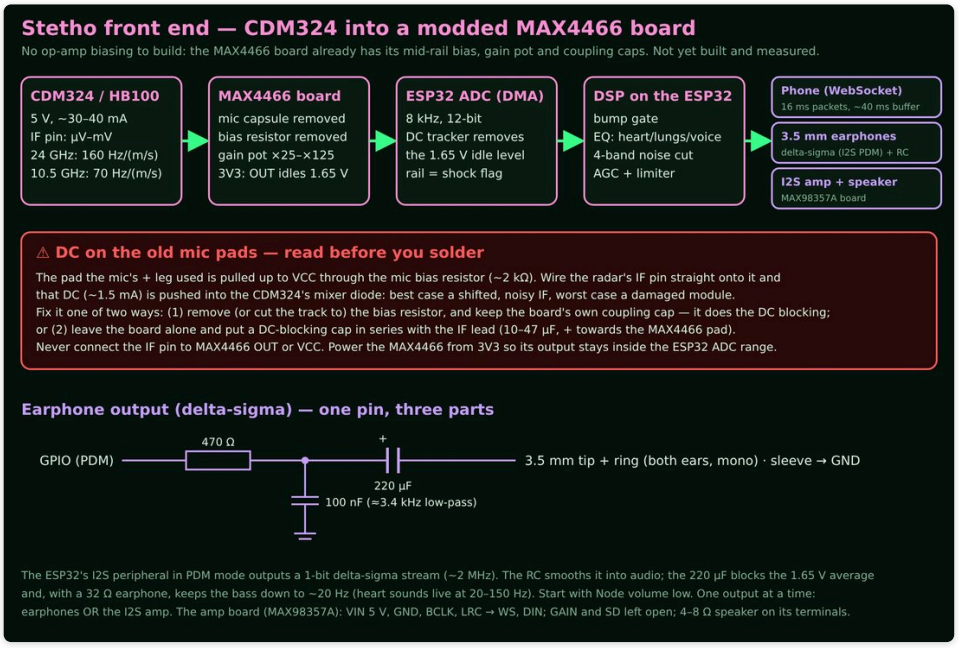
CHAPTER 5

Stetho front end: the MAX4466 mod

hardware/max4466-mod.md

Not a medical device. This is a physics experiment: listen to the tiny vibrations a CW Doppler radar picks up from a chest, a throat or a moving person.

Status: worked out on paper and in the firmware, **not yet built and measured**. The component values on the MAX4466 boards vary between sellers, so check yours (below).



Why a MAX4466 microphone board

You don't want to build op-amp biasing by hand, and you don't have to. The common "MAX4466 electret microphone amplifier" board (Adafruit #1063 and many GY-MAX4466 copies) is already a complete single-supply audio preamp:

- the op-amp's input is biased at mid-rail, so the output idles at VCC/2 (1.65 V on 3V3), right in the middle of the ESP32 ADC's range;
- gain is adjustable with the little pot (about ×25 to ×125);
- the gain-setting leg is AC-coupled, so DC gain is ~1 and a DC offset on the input can't slam the output into a rail;
- it runs from 2.4–5.5 V, so you power it from the ESP32's 3V3.

Is it OK for this? Yes for what the Stetho tab now does (heart sounds at 20–150 Hz, throat/larynx buzz, breath hiss, the Doppler "warble" of people moving). The board is an audio amp, so it has a low-frequency corner (the cap in its gain leg) somewhere around tens of Hz. Below that corner it only passes signal at ×1. That's fine for sound. The slow chest *motion* (breathing at 0.2 Hz, the heartbeat's 1–2 Hz movement) then reaches the ADC with little gain, which is why the vitals estimates may be weak. Breathing is a big signal and should still show. See "Lowering the corner" if you want more of it.

What it won't do on its own is give the ×1000+ gain a far-away subject needs. If the signal is too small with the pot at maximum, chain a second modded board (see below).

The mod

1. **Desolder the electret mic capsule.** Its two pads are where the radar's IF goes in.
2. **Remove the mic bias resistor** (or cut its track). This is the resistor (typically ~2 kΩ) that runs from VCC to the mic's + pad and powers the capsule. The radar must not see it — see the warning.
3. **Keep the board's input coupling capacitor.** It sits between that pad and the MAX4466 input, and it is what blocks DC between the radar and the amp.
4. Wire up:

From	To	Note
CDM324 / HB100 **IF**	old mic **+** pad (the one that went to the bias resistor)	short, twisted with GND, or thin coax
CDM324 GND	MAX4466 GND and old mic **−** pad	one common ground
CDM324 VCC	ESP32 board 5V	~30–40 mA

From	To	Note
MAX4466 VCC	ESP32 **3V3**	so OUT can't exceed the ADC range
MAX4466 OUT	ESP32 ADC pin (C3: GPIO1, S3-Pico: GPIO7)	

5. Set the gain pot to the middle. In the phone's Settings, turn **Stetho** on and watch the Stetho tab. If the waveform shows "rail" or clipping, turn the pot down.

> **⚠ DC on the old mic pads — read before you solder**
>
> The mic's + pad is pulled up to VCC through the mic bias resistor. If you wire the radar's IF pin straight onto it and leave that resistor in place, about 1.5 mA of DC is pushed into the CDM324's IF pin, which is the mixer diode's output. At best you get a shifted, noisier IF. At worst you damage the module.
>
> Fix it **one** of two ways: 1. **Remove the bias resistor** (recommended) and keep the board's own coupling cap, which does the DC blocking; or 2. leave the board alone and put a **DC-blocking capacitor in series with the IF lead**: 10–47 µF, with + towards the MAX4466 pad (that side sits near VCC). The bias resistor then loads the IF a little, so expect slightly less signal.
>
> Never connect the IF pin to MAX4466 OUT or VCC. Never run the MAX4466 from 5 V when its OUT goes to the ESP32: the idle level would be 2.5 V and a loud signal would drive the pin past 3.3 V.

If the Stetho tab shows nothing

The box at the top of the Stetho tab says where it stops: - **"Stetho is off"**: press *Turn it on* (or Settings → Stetho channel → On). - **"could not start"**: the reason follows; usually the pin isn't an ADC1 pin in your build. - **"reads 1.6 V... almost flat"**: the MAX4466 is fine but nothing is coming in. Wave a hand a few cm from the radar, check the IF wire and the common ground, turn the gain pot up. - **"stuck near 0 V"**: OUT isn't reaching the pin, or the board has no 3V3/GND. - **"stuck near the top"**: the MAX4466 is on 5 V (move it to 3V3), or the pin is damaged. On the C3 Super Mini GPIO3 or GPIO4 can take over (rebuild with `-DPIN_STETHO_I=3`); on the S3-Pico any of GPIO1–10.

With a multimeter, MAX4466 OUT should idle at about 1.65 V with the radar still. The serial monitor (115200) prints a `stetho:` line every 5 s with the same numbers.

Check your board's values

Sellers differ, and I could not get a readable copy of the reference schematic from here, so measure your board rather than trust a number: - **Input coupling cap**: in series between the mic + pad and the chip's input. It is usually 1 µF or more. - **Gain-leg cap**: in series with the small resistor that runs from the chip's − input towards ground or mid-rail. Together they set the low-frequency corner: f ≈ 1 / (2π·R·C). For example, 1 kΩ with 4.7 µF gives about 34 Hz.

Lowering the corner (optional, for more chest motion)

Replace the gain-leg cap with a bigger electrolytic (e.g. 100–220 µF, + towards the op-amp input side): with 1 kΩ that moves the corner to ~1 Hz or below. The board then takes a few seconds to settle after power-up. Heart *sounds* are unaffected; slow chest motion gets far more gain. Keep the input coupling cap ≥ 1 µF.

More gain: two boards in a row

Mod a second board the same way (mic **and** bias resistor removed). Wire the first board's OUT to the second board's old mic + pad, and the second board's OUT to the ESP32. Both boards run from 3V3, with a common GND. Each board's own coupling cap blocks the previous board's 1.65 V. Start with both pots low.

HB100 instead of CDM324

Same wiring (HB100 pins: +5V, GND, IF). It runs at 10.525 GHz, so the phase swing for the same movement is about 2.3× smaller (weaker heart signal). But it **does not clash with the RD-03D** (see DESIGN_NOTES.md).

Ready-made alternatives

- **MAX9814 boards** have their own automatic gain control. Avoid them here: their AGC pumps on every knock and fights the AGC on the ESP32.
- **"HB100/CDM324 Doppler backpack" boards** (e.g. limpkin's, now retired) are tuned for speed measuring. They output a frequency/square wave and have a low upper cutoff, so they are not suited to listening.
- **KY-037/KY-038 "sound sensor" boards** are comparators, not linear amps. Not suitable.

If you'd rather build your own amp

The previous version of this page proposed two op-amp stages (MCP6002, ×100 then ×30–100, mid-rail bias, 0.05 Hz–1 kHz). It's still valid if you want the full vitals band with lots of gain. Keep the output on 3V3, with a 1 kΩ series resistor and clamp diodes before the ADC.

Firmware side

- The ADC runs in DMA mode at 8 kHz on `PIN_STETHO_I` (`firmware/platformio.ini`). A ~2 s DC tracker removes the 1.65 V idle level.
- Processing on the node (`firmware/include/audio_dsp.h`): bump gate, preset EQ, 4-band noise suppressor, AGC and limiter.
- Outputs: the phone (HOW_IT_WORKS.md), 3.5 mm earphones and an I2S amp (audio-out.md).

CHAPTER 6

Sound out of the node

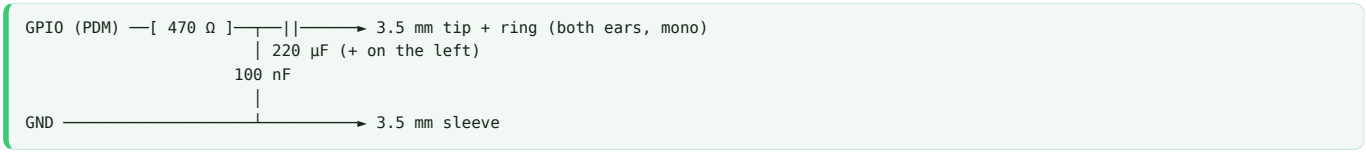
hardware/audio-out.md

The cleaned-up Stetho audio can play straight from the ESP32. Choose **one** output at a time in the Stetho tab under *Sound out of the node itself*. Both work with no phone connected. The last setting is remembered.

Status: firmware compiles; not yet tried on hardware.

1. 3.5 mm earphones: delta-sigma on one pin

The ESP32's I2S peripheral, in PDM mode, puts out a 1-bit delta-sigma (pulse-density) stream of about 2 MHz on a single pin. This is the "pseudo-PWM" audio you asked for. An RC low-pass turns it back into audio. The peripheral is DMA-clocked, so there is no timer jitter, unlike bit-banging a GPIO.



Board	PDM pin	(PDM clock, leave unconnected)
ESP32-C3 Super Mini	GPIO10	GPIO3
Waveshare ESP32-S3-Pico	GPIO16	GPIO13

- 470 Ω + 100 nF form a ≈3.4 kHz low-pass, just under the 4 kHz audio band at 8 kHz sampling.
- The 220 µF blocks the 1.65 V average. With 32 Ω earphones it keeps bass down to ≈23 Hz, and heart sounds live at 20–150 Hz. A smaller cap thins the sound.
- The volume is modest and plenty for earbuds. **Start with Node volume low.** If you plug it into a powered speaker's AUX input instead, keep the cap.
- Wired earbuds with an in-line mic (TRRS) work too: tip and ring are the ears.

2. I2S amplifier board (MAX98357A and friends)

A ~£3 MAX98357A board takes digital I2S and drives a 4–8 Ω speaker at up to ~3 W.

Amp pin	C3 Super Mini	S3-Pico
VIN	5V	VBUS
GND	GND	GND
BCLK	GPIO3	GPIO13
LRC	GPIO4	GPIO14
DIN	GPIO5	GPIO15
GAIN, SD	leave open	leave open

The firmware sends the same sample to left and right, so the board's default "(L+R)/2" mode and its "left only" mode both play it at full level.

"I2C amp": cheap amp boards are usually I2S (three wires: BCLK, LRC, DIN), not I2C. If yours is an analogue class-D board (PAM8403 and the like, with L/R/GND audio inputs), feed it from the earphone circuit above instead.

Feedback warning: a speaker near the radar module will be heard by the radar (the cone moves!). That's a loop. Point the speaker away, keep the volume down, or use earphones.

Pins come from one place

All pins are build flags in `firmware/platformio.ini`; the diagrams in `docs/images/` are drawn from them by `tools/draw_illustrations.py`. The S3-Pico's GP2–GP5 = GPIO13–16 mapping follows Waveshare's numbering pattern. Check it against Waveshare's pinout before soldering.

CHAPTER 7

Changelog

CHANGELOG.md

All notable changes to Blip. Semantic versioning; `-wip` marks pre-releases where many features may be broken.

[0.2.2-wip] — 2026-11-13 (OLED auto-scan, Track 1/3 on the Radar page, first RD-03D walk-around)

- **OLED is found automatically at boot.** The default is now **Auto-detect**: the node scans the I2C bus for a display at 0x3C, then 0x3D, and starts it if one answers (the log says where, or that nothing did). A bare SSD1306 cannot be asked its size over I2C, so Auto assumes **128×64**; for a 0.91" 128×32 panel pick it once in Settings and it is remembered. Choosing a size or Auto in Settings re-scans straight away, no reboot. Nodes that already had a size saved keep it. Not yet re-tested on the author's hardware after this change.
- **Track 1 / Track 3 switch on the Radar page**, next to the cards. It is the same setting as Settings > Radar tracking mode (the two stay in step) and sends the RD-03D its single/multi command. Multi stays the default. Checked in a browser against the simulator; not yet on a real board.
- **First real-hardware results (RD-03D, author's bench, 2026-11-12):** the radar tracks up to three targets in the default multi mode, and it also tracks big metal objects (fridge, chair) as the author walks towards them: the module reports anything whose range is changing relative to it. In a 5 m x 5 m house, with walls close in every direction, it is noisy; an outdoor walk is the next test. The OLED showed nothing at first because the size defaulted to None; picking 128×64 in Settings fixed it (that is what Auto now does). The LD2450 arrives later and has not been tried.
- Firmware images for all six envs rebuilt (see `firmware/prebuilt/BUILD_LOG.md`). Tests: parser, tracker, flasher, launcher, soak and OLED-render tests pass; page switch checked with Playwright against `tools/sim_radar.py`.

[0.2.2-wip] — 2026-11-10 (Stetho abandoned, kept optional)

- **Stetho (Doppler stethoscope) is parked as abandoned and stays an optional install.** Nothing is removed: the firmware channel, page tab, tests and docs still build and run. Reason, from the author: *"Needing to build an ultra-low-noise, ultra-high-gain, fast-slew-rate instrumentation amplifier is just too much work for a 'fun' 30-minute project, but I encourage anyone gifted enough with analogue, or mad enough, to carry on my research 😉"* Bench findings are in `docs/STETHO_SIGNAL_NOTES.md`: the static was supply noise (fixed by 100 uF caps), the heartbeat at 2 cm stays at the noise floor.

[0.2.2-wip] — 2026-11-08 (static, post-bump dropout, pattern recognition)

Everything below is verified on a PC with synthetic signals only. **Nothing here has been heard on real hardware yet**; see `docs/STETHO_SIGNAL_NOTES.md` for the A/B procedure. - **Fix: half a second of lag and heavy attenuation after a bump.** The old chain muted by 30 dB for 150 ms plus a 120 ms release, and the AGC and noise-floor trackers kept learning from the knock, so the gain stayed 15 to 48 dB too low for seconds afterwards. Now: a shallow duck (5 to 18 dB, scaled by the Bump rejection slider, 60 ms hold, 25 ms release); a 2 ms look-ahead peak limiter so a knock is limited and not muted; the AGC, level and noise-floor estimators freeze during a bump or a detected movement (300 ms tail, 3 s cap); and a level-excursion detector that holds the AGC and speeds up the level decay even when neither detector fires. Tests: signal back within 4 to 9 dB one beat after a knock and within 2 dB four beats after; a knock never exceeds the limiter ceiling. - **Burst blanker (Heart and Lungs presets).** A noise-blanker for short loud bursts above the preset's passband (the "5 to 10 heavy breaths a second" static fits the Wi-Fi beacon every 102.4 ms, which is a hypothesis, not a measurement). It watches the band above the passband, and when a burst far above its slow reference appears it gates the look-ahead delay path for at most 12 ms, then waits 150 ms before it can fire again. Off for Voice, Wide and Raw. Switch: Stetho tab, Experiments, Burst blanker. - **Averaged ADC sampling (experiment).** 2, 4 or 8 readings per output sample with the extremes dropped, to remove ADC spikes before they are heard. Default stays at 1 until it has been compared on hardware. Stetho tab, Experiments, ADC sampling (or `/api/lab?os=4`). - **Heart and breath pattern recognition in the page** (`CycleTracker`): autocorrelation to find the repeat period (walking down from multiples so 72 bpm is not read as 36), then folding the signal on that period for a fine rate, a beat template and a lock quality. Rates show `?` when the rhythm is only roughly steady and nothing when there is no pattern. Replaces the zero-crossing breath estimate and the first-autocorrelation-peak heart estimate. Synthetic results: 48 to 140 bpm within 2.5 bpm, noise-only and irregular knocks never reach a full lock, breath rate within 1 per minute from chest motion where the old estimate was off by 2 to 28. - **Beat lock (optional, off by default):** once a heart or breath rhythm is locked, the sound between beats is turned down by 9 dB (heart) or 6 dB (lungs). Test: the heart sounds change by less than 1 dB and the gaps drop 5 to 7 dB. - **Record 10 s to WAV** button (8 kHz, 16-bit, named with preset, averaging and blanker settings) and `tools/analyze_wav.py`, which reports clipping, band energy, whether short bursts repeat on the Wi-Fi beacon interval, and the heart or breath rhythm. This is how to find out what the static really is. - Tests added: `tests/test_dsp_dynamics.cpp` (sanitizers, in CI), `tests/test_vitals_pr.js` (Node, in CI), `tests/test_analyze_wav.py`. The RV32 cost check with the new chain: about 7.7% of the C3 core (11.6% at 1.5 cycles per instruction).

[0.2.2-wip] — 2026-11-07

- **Stetho stability test lab (no board needed for most of it).** `tools/soak_test.py` is a stdlib-only soak/bisect harness: it opens the same WebSocket the app does, counts audio packets and gaps, watches the node's boot counter for resets, and flips the new runtime **Lab** switches (`/api/lab`: pacing of `loop()`, DSP bypass, send policy, synthetic audio source instead of the ADC, text pushes, quiet logging, skipped subsystems, audio task priority) so ONE flash can isolate the freeze by trial instead of one reflash per theory. `tests/test_soak.py` tests the harness against `tools/sim_radar.py` (which now emulates the lab, 503-while-down and a crash). `tests/test_dsp_stability.cpp` runs the DSP chain through hostile scenes under UBSan/ASan. `tests/rv32_cost/run.py` compiles the audio path for RV32IMC, runs it in an emulator and counts instructions: Stetho costs about 7% of the C3 core (10% at 1.5 cycles/instruction), so plain CPU starvation by the DSP is ruled out. See `docs/STETHO_STABILITY.md`.

- **Fix: the bump gate could stay latched.** A sustained loud scene held the gate's slow-envelope tracker frozen indefinitely (a measured 10 s before release). The freeze is now capped at 600 ms; release is within 750 ms. Test added.
- **Fix: status JSON length arithmetic** could underflow on a long string; appends are now clamped (`jcat`) in a 2 KB buffer.

- The async_tcp freeze itself is **not** claimed fixed; the Lab build is how to find it.

- **Fix: stopping Stetho crashed the node.** A bench-test log decoded against its exact build showed `assert failed: xTaskPriorityDisinherit` from `adc_lock_release` from `adc_continuous_stop` from `Stetho::adcEnd()` . The ADC driver holds a mutex for as long as it samples, and FreeRTOS asserts if a different task gives it back: the ADC was opened by the web/loop task and closed by the audio task, so every stop (Stetho switched off, auto-sleep, a failed start) reset the board. The audio task now opens and closes the ADC itself, `start()` waits for it to report whether it came up, and its stack is 8 KB. This is not the `async_tcp` crash, but it is a real bug on the same feature.

- **Bench test stage 2 now streams to a real client.** The first bench log ran the full 2 minutes, twice, with Stetho sampling and a phone joined to the hotspot and no crash, but a bare Wi-Fi join opens no WebSocket, so no Stetho audio was sent at all. Stage 2 now serves a small page at `http://192.168.4.1/` that opens a WebSocket and counts the audio, the timer starts when that page connects, and each second's line shows pages open and packets sent/skipped. That is the load the app has when it crashes. The PASS line is printed before Stetho is stopped.
- **A/B test: the old Stetho send logic is back.** Stetho stopped working completely after the latency rework. The rework called `ws.binaryAll()` on every audio and vitals packet and let a smaller per-phone queue (8 instead of the library's 32) refuse them. That kept `loopTask` inside AsyncWebSocket's client and queue locks, and lwIP's `tcp_write` , dozens of times a second: the same locks `async_tcp` needs, and where the decoded crash found `loopTask` . This build puts back the send logic that ran for minutes on a real C3: send only when every phone has room (`availableForWriteAll`), only the newest two packets per pass, throw away a backlog of six (~190 ms). The queue cap is back to the library default (32). Mu-law 32 ms packets are kept. If Stetho runs again, the send path was the cause.
- **Bench test in the flasher:** a separate 🩺 **Diagnostics** tab (next to ⚡ *Flash Blip*) flashes the bundled bench image for the chosen board (envs renamed `c3-supermini-bench` / `s3-pico-bench` , images in `firmware/prebuilt/` , built by CI). New stage-1 checks: **fake flash** (read-only address wrap-around test and the chip's own capacity ID) and a **PSRAM** size and address-pattern walk on the S3-Pico. The RAM march now tests on-chip SRAM only. Stage 2 sends exactly as the app now does.
- **Standalone bench test** (`firmware/bench/` , envs `bench-c3` / `bench-s3`): a separate firmware, not the Blip app, for checking a board before you wire it up — in case a bad batch is part of the crash above. Stage 1 runs ~15 s of self-checks with no wiring needed (chip identity, a RAM march test, a flash/NVS scratch write, a Wi-Fi scan + AP bring-up, a Stetho-pin ADC read, an I2C scan, a UART loopback that's skipped rather than failed if nothing's jumpered). Stage 2 then runs the real Stetho sampler against the real Wi-Fi AP + web server for 2 minutes with a phone or laptop joined — the exact combination that has been crashing some boards — and if the board resets instead of finishing, the next boot reports it as failed and how far it got, using an RTC marker that survives the reset. See `firmware/bench/README.md` .
- **The disconnect-burst fix did not stop the crash.** A fresh node log showed the same `async_tcp` watchdog reset with no Wi-Fi client ever leaving or rejoining nearby — it just needs Stetho running and a client present, 4-9 seconds later. That rules out the previous theory. The failed-allocation hook never fired either, which rules out running out of memory. What's left points at CPU time, not memory: on this single-core chip, Stetho's sampling task and the Wi-Fi/networking stack are competing for the same core, and something about it is starving the networking task for several seconds straight. The health line now runs every 1 second (not 10) while Stetho is on, since the last two crashes happened faster than the old interval could ever catch twice, and any single `loop()` pass over 150 ms is now logged the instant it happens rather than waiting for the next health tick. This build does not claim to fix the crash — it's aimed at finally catching it in the act.
- **Found the real crash, from a user's node log (superseded above):** a C3 froze and watchdog-reset a few seconds after Stetho started, next to a client joining/leaving the hotspot. Decoding the crash addresses against the exact build showed `loopTask` inside `AsyncWebSocket` 's own internal client-list lock — the same lock the library takes while handling a client disconnect. Stetho's drain loop (`while` over every queued audio packet) could fire a burst of `ws.binaryAll()` calls back to back right at that moment. The drain is now capped at 4 packets per `loop()` pass, spreading any backlog over a few ticks instead of bursting it. Also added: a hook that logs the exact size and caller of the *first* failed heap allocation (so a crash has a concrete cause, not just a watchdog panic), and `async_tcp` 's own state (Running/Ready/Blocked) and stack headroom in the health line, so a future crash log shows what the networking task was doing, not just that it stopped checking in.
- **Node log** (like Skipper Flipper's): timestamped lines kept in 4 KB of RTC memory, which survives a crash or restart (not a power cut). The log has the start reason, Wi-Fi join/leave with reason codes, pages connecting and disconnecting, Stetho start/stop, and a **health line every 10 s**: free memory (lowest, biggest block), devices on the hotspot, open pages, requests and captive-portal probes, loop time and the Stetho counts. See it under Settings → *Node log* (Refresh / Save as file), at `/api/log` , or over USB with the flasher's new 🔌 **USB log** button (no flashing). This is to find the memory drain seen on a C3 (139 KB → 23 KB free within a couple of minutes, then the node stops answering).
- **Wi-Fi transmit power setting** (8.5 / 15 / 19.5 dBm). It defaults to 8.5 dBm on the ESP32-C3 Super Mini, whose weak antenna makes Android phones struggle to join at full power, and 15 dBm on S3 boards.
- The WIP banner shows the **build time**, so you can tell which build is running even when the version number is unchanged.
- **Fix: the node froze a few seconds after Stetho started** (web server dead, no restart), seen on a real C3. The separate WebSocket sender task from the earlier 0.2.2 test builds is gone, and all sending is back in `loop()` , which ran for minutes in the earlier test. Kept: mu-law 32 ms packets, the 8-message queue cap per phone and the 380 ms node buffer. The page says *Stetho is starting…* instead of a false *could not start* in the moment after it is switched on.
- **Crash/stall diagnostics:** the node reports why it last restarted (power on, crash, watchdog, brown-out) and how many times it has started; the Stetho box shows it with the uptime, and says "No update from the node for N s" with the page's reconnect count when updates stop. The serial monitor prints the same at start-up.
- **Stetho audio sent as mu-law in 32 ms packets** (`'U'` packets, 256 bytes): half the bytes and a quarter of the messages of the old 16 ms int16 packets. The status is pushed every 2 s, and the page falls back to asking over HTTP if pushes stop. "No reading yet" shows as *measuring…* instead of "−0.00 V". The page still plays the old `'A'` packets.
- **Firmware build log** `firmware/prebuilt/BUILD_LOG.md` : `make_prebuilt.py --note` records each bundled build (version, time, source hash, SHA-256 per image). Only the last 3 builds' binaries are kept in git history, to keep the repo small.
- **The launcher opens the Blip Flasher first:** a red "maroon radar" page at `/` that is only the flasher, with a small *Radar simulator ›* button. The simulator moved to `/sim` and has a ⚡ *Flasher* button back. The Flash tab inside the simulator is gone, to remove confusion. Phones on `--lan` and other hosts get the simulator, never the flasher.

- **Fix: Stetho audio lag (seconds) and the board's web server dying after a few minutes.** Seen on a C3: 84 packets sent, 7,567 dropped. Stetho packets now go out from their own task the moment they are made, instead of waiting for `loop()`. Each phone's WebSocket queue is capped at 8 messages (`WS_MAX_QUEUED_MESSAGES`), so a slow link skips audio instead of building up delay or memory. The node's status is pushed over the WebSocket once a second, instead of the page opening a new HTTP connection every second. Targets are only sent when a new radar frame arrives (1 Hz when there is no radar). The OLED is probed at start and switched off if nothing answers, so a missing display can't stall the node. USB serial output never waits for a PC that isn't reading. The Stetho tab shows free memory, the slowest loop and uptime, so a leak would show. Not yet re-tested on hardware.
- **Stetho diagnostics:** the Stetho tab now says why nothing is showing: Stetho off (with a *Turn it on* button), asleep, the ADC failed to start (with the reason), no samples yet, or the live voltage on the ADC pin with hints for "stuck near 0 V", "stuck at the top (MAX4466 on 5 V?)" and "flat, no signal". It refreshes every second while the tab is open. The serial monitor prints a Stetho health line every 5 s. Status JSON gains `stetho.err/samples/invalid/sent/level`. The audio task's stack is raised to 6 KB and a failed task start is reported.
- Fix: starting the simulator while an old one was still running crashed with "Address already in use". It now stops the old Blip simulator and takes over (or uses the next free port if another program holds it), and prints this computer's network address. `--lan` also listens on the network so a phone can open it.
- **HLK-LD2450 option** (`-DRADAR_LD2450=1`, envs `c3-supermini-ld2450` / `s3-pico-ld2450`): same 30-byte frame, so the same parser; no RD-03D mode commands are sent; Single mode is picked in firmware (`rd03d::keep_nearest`). Status reports `model`. Native tests added. Not yet tested on hardware.
- Fix: a `volatile` decrement warning in the Stetho mute counter.
- **Flash tab** in the simulator page replaces the three `Flash-*` scripts: pick board, radar and port, press one button; five-step progress, upload percentage, colour-coded log with *Copy*, plain-English tips for the usual failures, serial output afterwards, *Cancel*. Installs PlatformIO into `.venv` on the first run. Only answers this computer (loopback + Host check + per-run token); `--no-flash` hides it. `tools/flasher.py`, `tools/serial_tail.py`, `web/flasher.html`; tested with `tests/test_flasher.py` + `tests/fake_pio.py`.
- **Quick flash (default)**: the release now bundles prebuilt whole-chip images for all four builds in `firmware/prebuilt/` with a `manifest.json` (SHA-256, size, version, source hash). The Flash tab shows exactly what it will write and the exact esptool command, verifies the checksum first, says whether the bundle matches your source, and needs only esptool (~40 MB) — no Git, no toolchain. *Build from source* stays one click away. `tools/make_prebuilt.py` refreshes the bundle.
- Every GitHub release now carries a **PDF manual** (README, how it works, design notes, hardware guides, changelog, licence), built by `tools/make_pdf.py`; a copy is in `docs/`.
- Release workflow builds and attaches all four firmware variants (RD-03D and LD2450 for both boards).
- Fix: the scope stopped throwing errors when the window is resized while another tab is open.

[0.2.1-wip] — 2026-11-05

- **Fix:** the single/multi-target command is now wrapped in *enable configuration* / *end configuration*, 200 ms apart, non-blocking (as the original project does). Earlier versions sent the bare mode command, which the module probably ignored.
- New **Smart target filter** (experimental, opt-in): own track ids, alpha-beta smoothing, 3-of-5 confirmation, 1 s coasting, same-bearing/lockstep ghost test, learned clutter map with *Forget learned clutter*. `tracker.h` + native tests; Python copy for the simulator with a cross-check test. Scope gets *Show filtered*.
- **Auto-sleep** (Never/2/5/15/30 min) and an optional **CDM324 power switch** (`PIN_STETHO_PWR`) alongside the RD-03D one; 0.6 s mute after the CDM324 powers up.
- **OLED redesign:** each person is a circle with the distance inside (metres + small raised tenths digit, nearest solid), no status row, corner icons, IP only until a phone joins. Drawing code is shared with a PC renderer, so README previews are exact.
- Simulator room is now messy by default (noise, slot swaps, a wall reflection, a fan, flickers); `--clean` for the old perfect room.

[0.2.0-wip] — 2026-11-05

- **Firmware now compiles** for both boards (pioarduino platform pinned at 55.03.312 = Arduino-ESP32 3.3.12; libraries pinned by GitHub tag). Fixed: string build flags with spaces were mis-quoted in `platformio.ini`. C3 uses the 3 MB `huge_app` partition.
- Stetho front end is now a **modded MAX4466 mic board** (mic and bias resistor removed) — `hardware/max4466-mod.md`, with the DC-on-the-mic-pads warning.
- Stetho sampling moved to **ADC DMA at 8 kHz** and a dedicated audio task.
- New on-node **audio DSP** (`audio_dsp.h`, fixed-point): bump gate with 4 ms look-ahead, heart / lungs / voice / wide / raw presets, 4-band noise suppressor, AGC with noise-floor cap, limiter. Native test `tests/test_audio_dsp.cpp`; before/after WAVs in `examples/`.
- **Live low-latency audio to the phone**: 16 ms packets, TCP_NODELAY, Wi-Fi sleep off, stale packets dropped; phone ring-buffer player (~40 ms) in an AudioWorklet or ScriptProcessor. Chest-motion packets at 100 Hz for the trace and rate estimates.
- **Node audio outputs**: 3.5 mm earphones via I2S PDM (delta-sigma) + RC filter, or a MAX98357A I2S amp — `hardware/audio-out.md`.
- Optional **radar power switch** (`PIN_RADAR_PWR`) to silence the RD-03D while a CDM324 listens; research on the 24 GHz clash in DESIGN_NOTES §11.5.
- Removed the reserved Stetho Q pin and the old two-stage op-amp page.

[0.1.0-wip] — 2026-11-05

Project started 2026-11-05 10:24. - RD-03D frame parser with sign-bit decoding and stream resync; native C++ and Python tests. - ESP32 firmware (ESP32-C3 Super Mini, Waveshare ESP32-S3-Pico): hotspot + captive portal, WebSocket/HTTP API, settings in NVS, optional SSD1306 OLED mirror (128×32 / 128×64), optional "Stetho" ADC channel. **Not yet compiled or run on hardware.** - Phone web UI: radar scope with trails and tracker ping, targets table, Stetho waveform with heart/breath estimates and sonification, settings, about, first-start disclaimer. - Python simulator with real RD-03D byte encoding, double-click launchers, flash helpers, illustrations drawn from the pin map, research notes.

CHAPTER 8

Third-party notices

THIRD_PARTY.md

Blip's own code is licensed under the PolyForm Noncommercial License 1.0.0 (see LICENSE.md). That licence covers only Matthew Armstrong's code and documents in this repository. Everything listed below keeps its own licence, and none of these projects or companies is affiliated with or endorses Blip.

Component	Used for	Licence (see their site)
arduino-esp32 (Espressif)	Arduino core for ESP32-C3/S3	https://github.com/espressif/arduino-esp32
ESPAsyncWebServer and AsyncTCP (ESP32Async fork)	HTTP + WebSocket server	https://github.com/ESP32Async/ESPAsyncWebServer
U8g2 (olikraus)	SSD1306 OLED driver	https://github.com/olikraus/u8g2
PlatformIO	build system	https://platformio.org
Python 3 standard library	simulator, tools	https://www.python.org
matplotlib, Pillow, Playwright	drawing the illustrations and screenshots (dev-time only)	their respective sites

Inspiration and reverse-engineering sources (no code copied): - Stevee87, *Arduino-ESP32-Radarproject* (MIT): https://github.com/Stevee87/Arduino-ESP32-Radarproject - Ai-Thinker Rd-03D specification and serial protocol documents. - Electronic Clinic's RD-03D write-up (a captured two-target frame is reused in the tests). - Papers cited in DESIGN_NOTES.md.

Trademarks (ESP32, Arduino, Waveshare, Ai-Thinker, Raspberry Pi Pico) belong to their owners.

CHAPTER 9

Licence

LICENSE.md

Required Notice: Copyright Matthew Armstrong (https://github.com/The-Dorkknight)

https://polyformproject.org/licenses/noncommercial/1.0.0

Acceptance

In order to get any license under these terms, you must agree to them as both strict obligations and conditions to all your licenses.

Copyright License

The licensor grants you a copyright license for the software to do everything you might do with the software that would otherwise infringe the licensor's copyright in it for any permitted purpose. However, you may only distribute the software according to Distribution License and make changes or new works based on the software according to Changes and New Works License.

Distribution License

The licensor grants you an additional copyright license to distribute copies of the software. Your license to distribute covers distributing the software with changes and new works permitted by Changes and New Works License.

Notices

You must ensure that anyone who gets a copy of any part of the software from you also gets a copy of these terms or the URL for them above, as well as copies of any plain-text lines beginning with `Required Notice:` that the licensor provided with the software. For example:

> Required Notice: Copyright Yoyodyne, Inc. (http://example.com)

Changes and New Works License

The licensor grants you an additional copyright license to make changes and new works based on the software for any permitted purpose.

Patent License

The licensor grants you a patent license for the software that covers patent claims the licensor can license, or becomes able to license, that you would infringe by using the software.

Noncommercial Purposes

Any noncommercial purpose is a permitted purpose.

Personal Uses

Personal use for research, experiment, and testing for the benefit of public knowledge, personal study, private entertainment, hobby projects, amateur pursuits, or religious observance, without any anticipated commercial application, is use for a permitted purpose.

Noncommercial Organizations

Use by any charitable organization, educational institution, public research organization, public safety or health organization, environmental protection organization, or government institution is use for a permitted purpose regardless of the source of funding or obligations resulting from the funding.

Fair Use

You may have "fair use" rights for the software under the law. These terms do not limit them.

No Other Rights

These terms do not allow you to sublicense or transfer any of your licenses to anyone else, or prevent the licensor from granting licenses to anyone else. These terms do not imply any other licenses.

Patent Defense

If you make any written claim that the software infringes or contributes to infringement of any patent, your patent license for the software granted under these terms ends immediately. If your company makes such a claim, your patent license ends immediately for work on behalf of your company.

Violations

The first time you are notified in writing that you have violated any of these terms, or done anything with the software not covered by your licenses, your licenses can nonetheless continue if you come into full compliance with these terms, and take practical steps to correct past violations, within 32 days of receiving notice. Otherwise, all your licenses end immediately.

No Liability

As far as the law allows, the software comes as is, without any warranty or condition, and the licensor will not be liable to you for any damages arising out of these terms or the use or nature of the software, under any kind of legal claim.

Definitions

The **licensor** is the individual or entity offering these terms, and the **software** is the software the licensor makes available under these terms.

You refers to the individual or entity agreeing to these terms.

Your company is any legal entity, sole proprietorship, or other kind of organization that you work for, plus all organizations that have control over, are under the control of, or are under common control with that organization. **Control** means ownership of substantially all the assets of an entity, or the power to direct its management and policies by vote, contract, or otherwise. Control can be direct or indirect.

Your licenses are all the licenses granted to you for the software under these terms.

Use means anything you do with the software requiring one of your licenses.